

Lecture 9: Basis of Image Segmentation

Dan Wu

danwu.bme@zju.edu.cn

Where we are?

时间	授课章次与主要内容	课外学习与作业要求
3.4	BIP绪论 介绍课程安排；基本概念；应用实例；理解视觉感知原理，了解数字图像的生成	课后阅读
3.11	BIP基础 掌握图像的矩阵表示、量化和采样，掌握像素和灰度；了解彩色图像的构成和基本处理思路	实验1
3.18	BIP空间运算 了解图像基本运算、定量测量；空间滤波、卷积操作，卷积核的设计；掌握直方图和灰度变化	作业1
3.25	图像空间增强 掌握几种常用的空间滤波和卷积方法；掌握常用的低通、高通空间滤波器设计；组合滤波设计	实验2
4.1	傅立叶域运算 空间傅立叶变换；离散傅立叶变换特性；快速傅里叶变换；图像小波变换	作业2
4.8	图像频域增强 空间傅立叶的特性；低通与高通频域滤波器	实验3
4.15	图像复原 图像复原处理的基本概念；噪声模型；空间滤波与频域滤波	作业3
4.22	形态学图像处理 形态学处理的基本操作；基本的形态学算法	实验4

Lecture 1-8: Basics of BIP

Where we are?

时间	授课章次与主要内容	课外学习与作业要求
4.29	图像分割基础 了解图像分割的定义，边缘连接与边界检测；掌握常用阈值的图像分割方法；区域分割算法	作业4
5.6	图像分割算法 聚类分割；分水岭算法、模版分割算法	实验5
5.13	助教答疑	
5.20	图像配准 医学图像配准定义；图像的变换模型；相似性测度；非线性配准；图像配准方法的评估	作业5 实验6
5.27	图像特征提取 边界特征描述；区域特征描述；放射组学	作业6
5.22	医学图像人工智能 机器学习的基本概念；深度机器学习和CNN	实验7
6.3	医学图像人工智能 新技术进展和应用实例	课程大作业
6.10	课程大作业	课程大作业

Lecture 9-16: Advanced BIP & Medical BIP

Basis of Image Segmentation

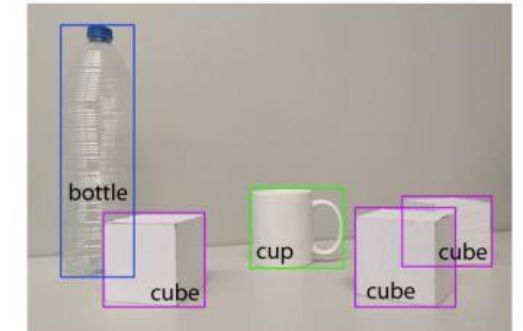
- Introduction in Image Segmentation
 - The state-of-the-art
- Basic image segmentation
 - Fundamentals: 1st / 2nd order derivatives
 - Point, line, edge
 - High-level edge detection
- Thresholding

Revisit – Goals of image processing

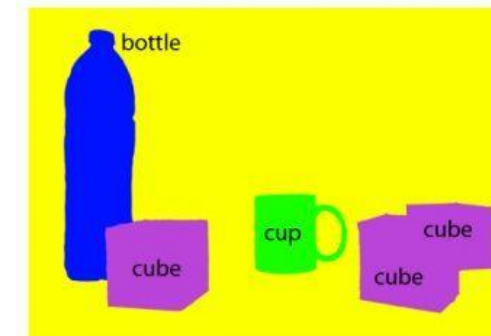
- Image processing – low level BIP
 - Improvement of pictorial information for human interpretation
(Improving the visual appearance of images to a human viewer)
- Image analysis – high level BIP
 - Analysis of scene data for autonomous machine perception
(Preparing images for measurement of the features and structures present)



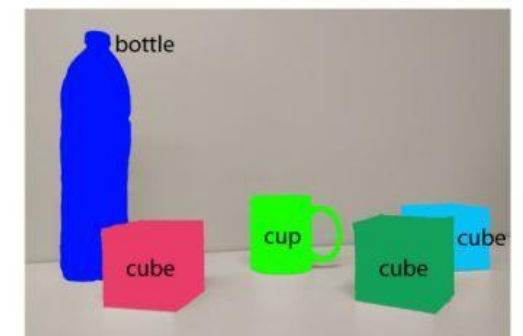
(a) Image classification



(b) Object localization



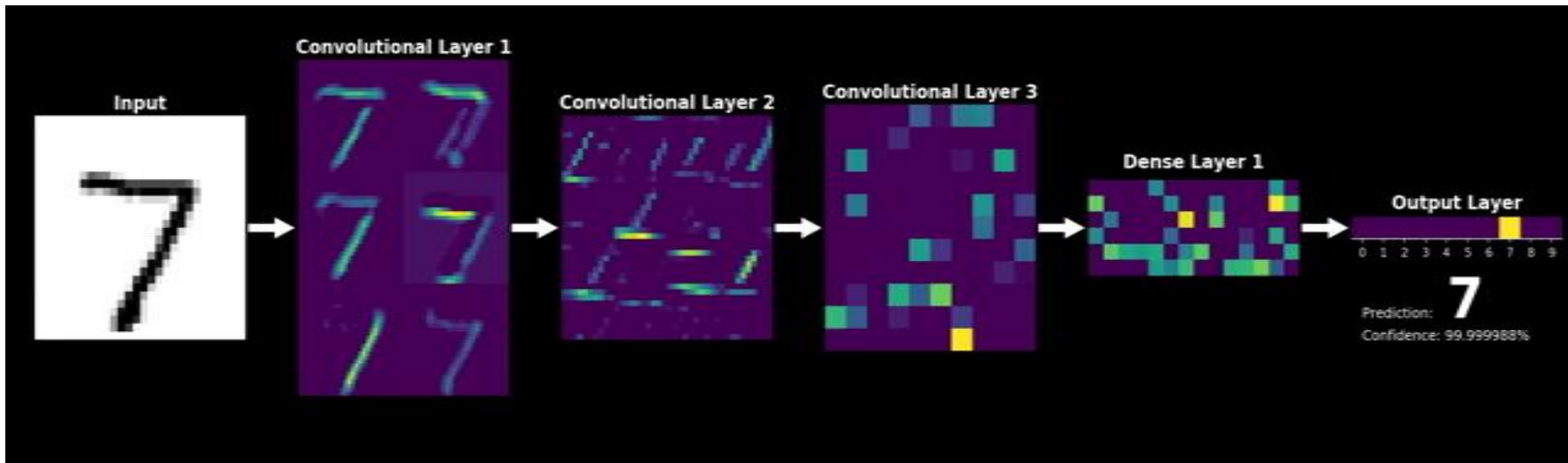
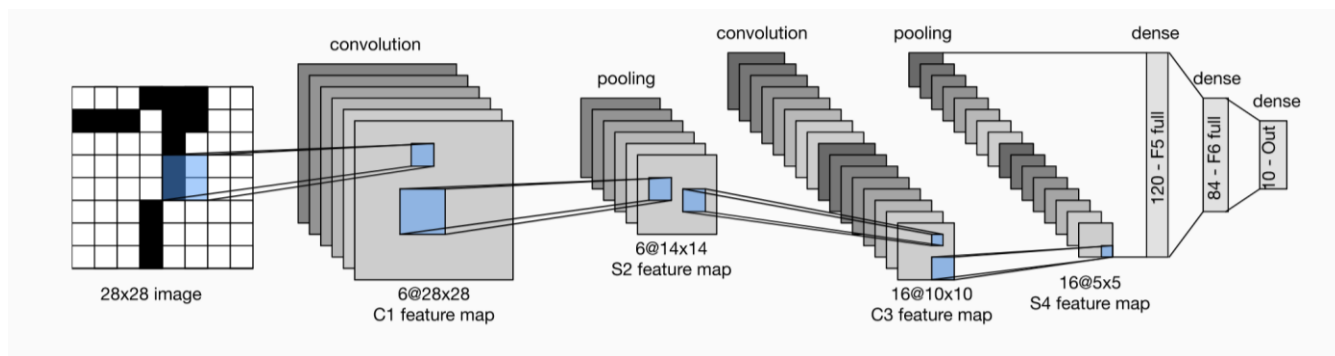
(c) Semantic segmentation



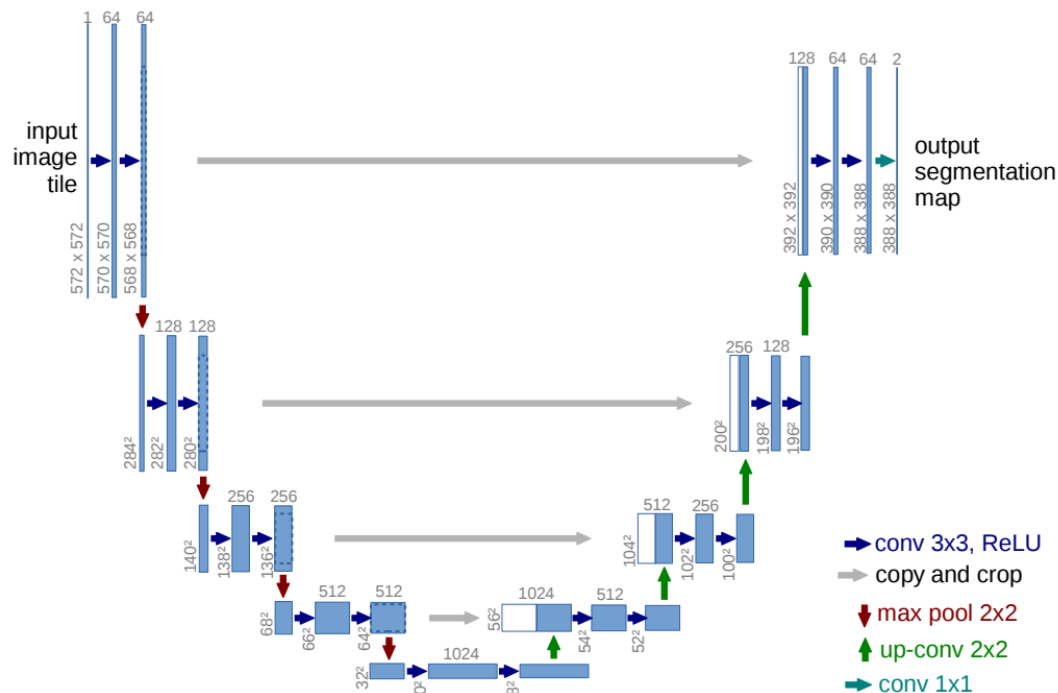
(d) Instance segmentation

Convolutional Neural Network

LeNet



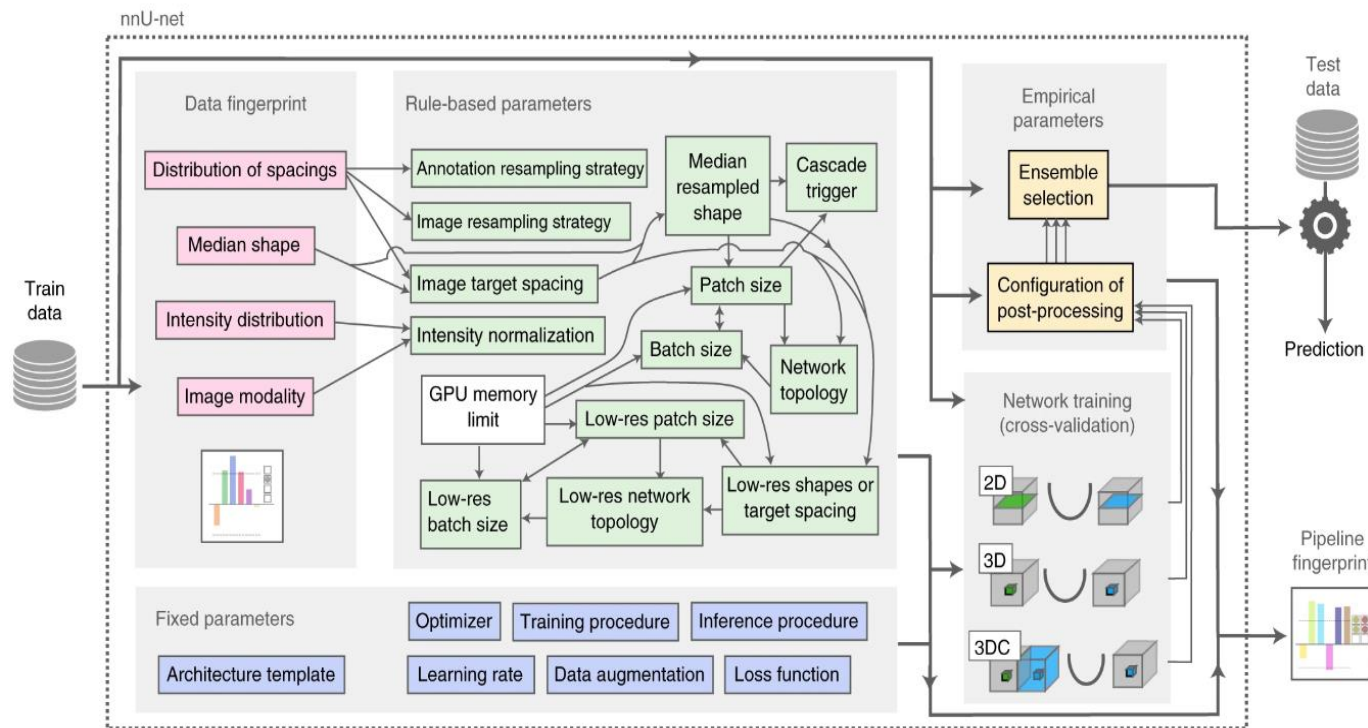
UNet



Ronneberger et al., MICCAI 2015

创新：一种基于卷积神经网络的U形架构，通过结合上下文信息和精确定位，实现了从极少量标注图像中高效学习

nnUNet



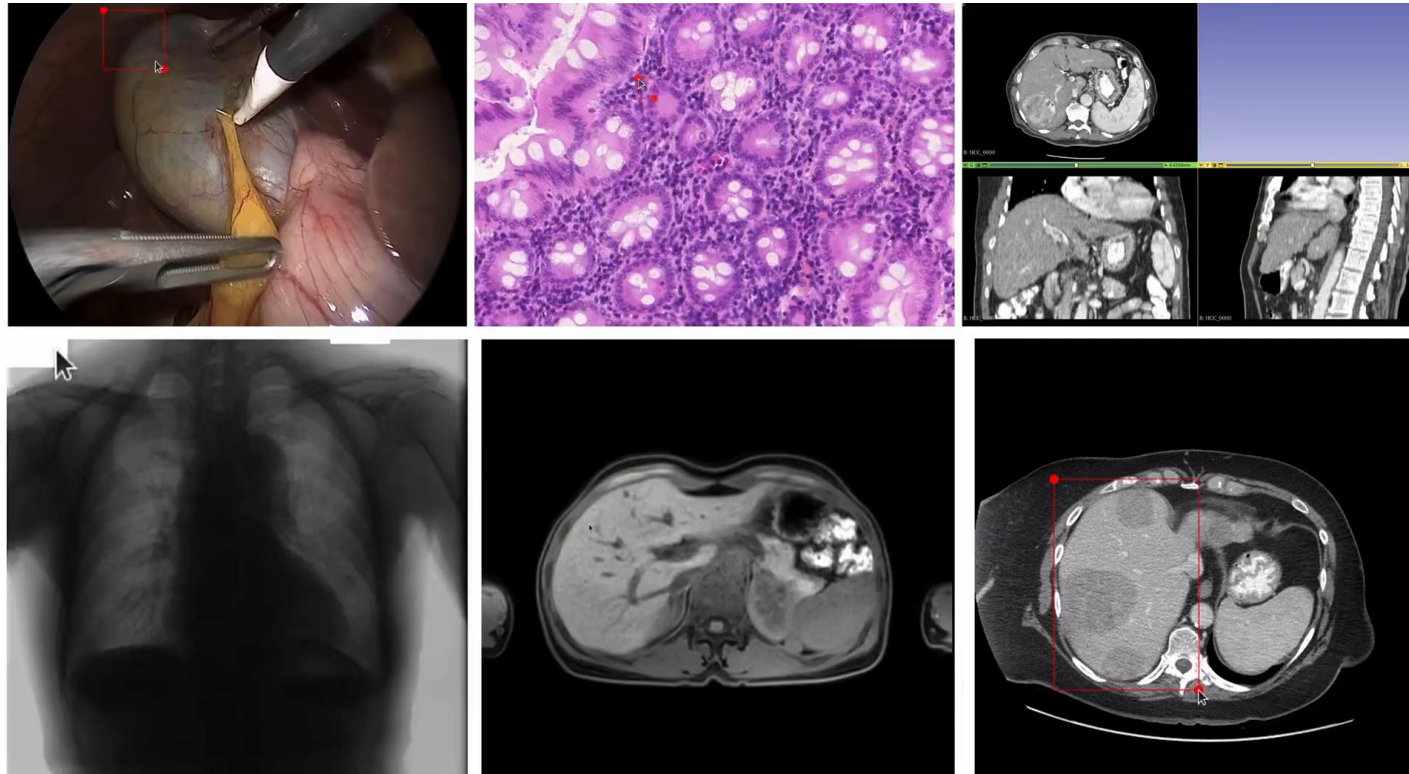
Isensee et al., Nature Methods 2020

创新：一种完全自动化的、自配置的深度学习方法，能够自动适配不同的数据集和分割任务，而不需要人工干预或手动调整超参数。

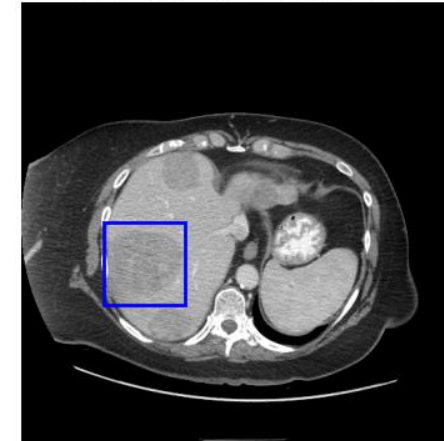
Segment Anything



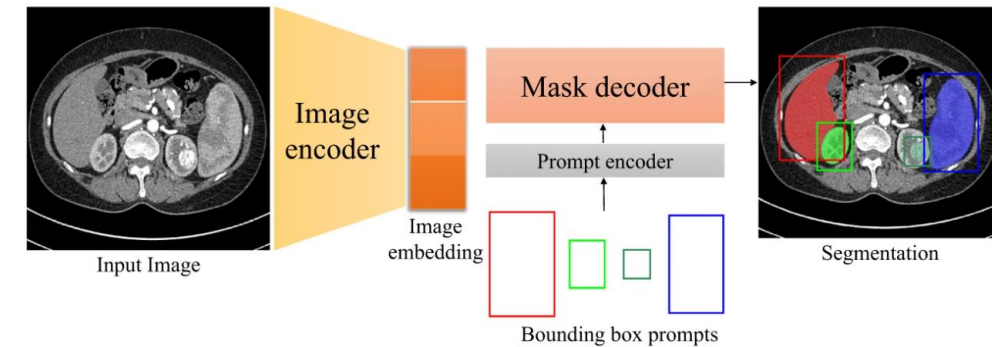
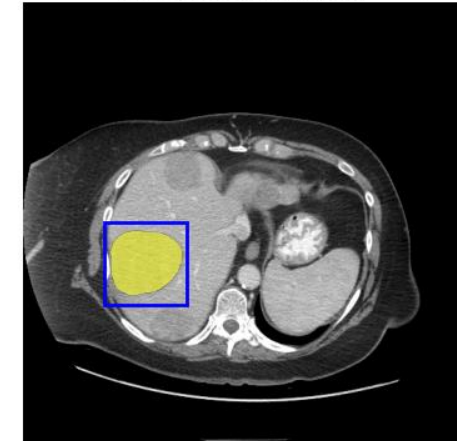
Segment anything in medical images (MedSAM)



Input Image and Bounding Box



MedSAM Segmentation



Basis of Image Segmentation

- Introduction in Image Segmentation
 - The state-of-the-art
- Detection of gray level discontinuities
 - Fundamentals: 1st / 2nd order derivatives
 - Point, line, edge
 - Higher level edge detection
- Thresholding

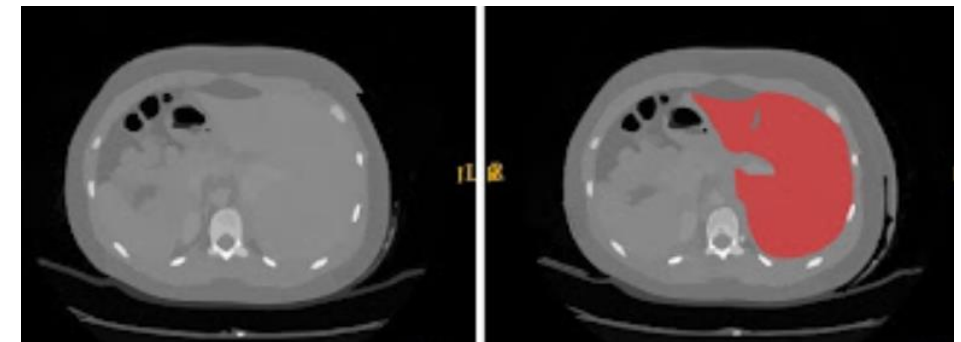
What is segmentation?

➤ Definition

- Subdivides an image into its constituent regions or objects $\bigcup_{i=1}^n R_i = R \quad R_i \cap R_j = \emptyset$

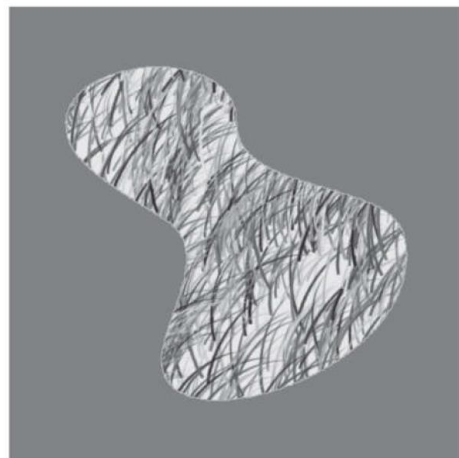
➤ Heavily rely on one of two properties of intensity values:

- Discontinuity – Partition based on abrupt changes in intensity
 - Point / line / edge / corner detection
- Similarity – Partition based on intensity similarity
 - Thresholding
 - Region growing / splitting / merging

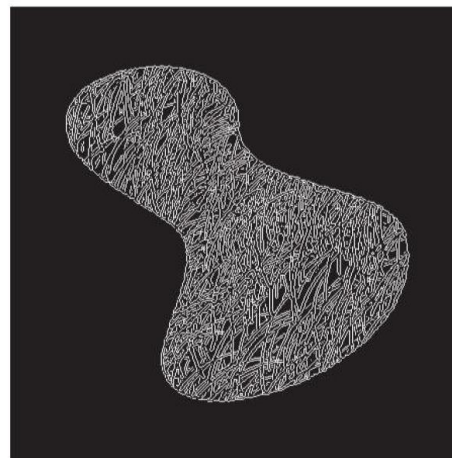
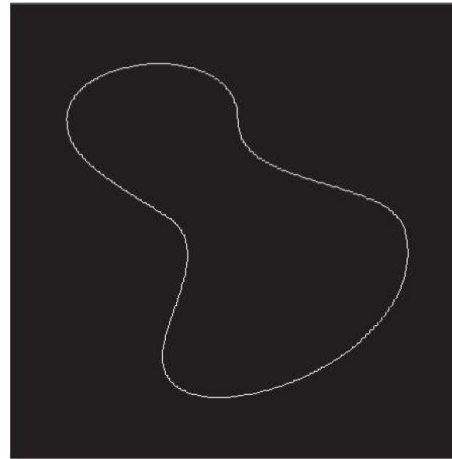


What is segmentation?

Texture



Intensity discontinuity



Segmentation



If $\sigma > 0$, label=1
within a 8x8
neighborhood

Fundamentals

➤ We want to extract 3 basic types of intensity discontinuity:

- Points
- Lines
- Edges

➤ What have we learnt in previous lectures to help us in this process?

- Convolution (or filtering)

$$g(x, y) = \sum_{s=-a}^a \sum_{t=-b}^b w(s, t) f(x + s, y + t)$$

Grayscale image

Mask coefficient

Fundamentals

➤ Approach

- First-order derivative

$$\frac{\partial f}{\partial x} = f(x+1) - f(x) = f(x) - f(x-1) = [f(x+1) - f(x-1)]/2$$

- Second-order derivative

$$\frac{\partial^2 f}{\partial^2 x} = f(x+1) + f(x-1) - 2f(x)$$

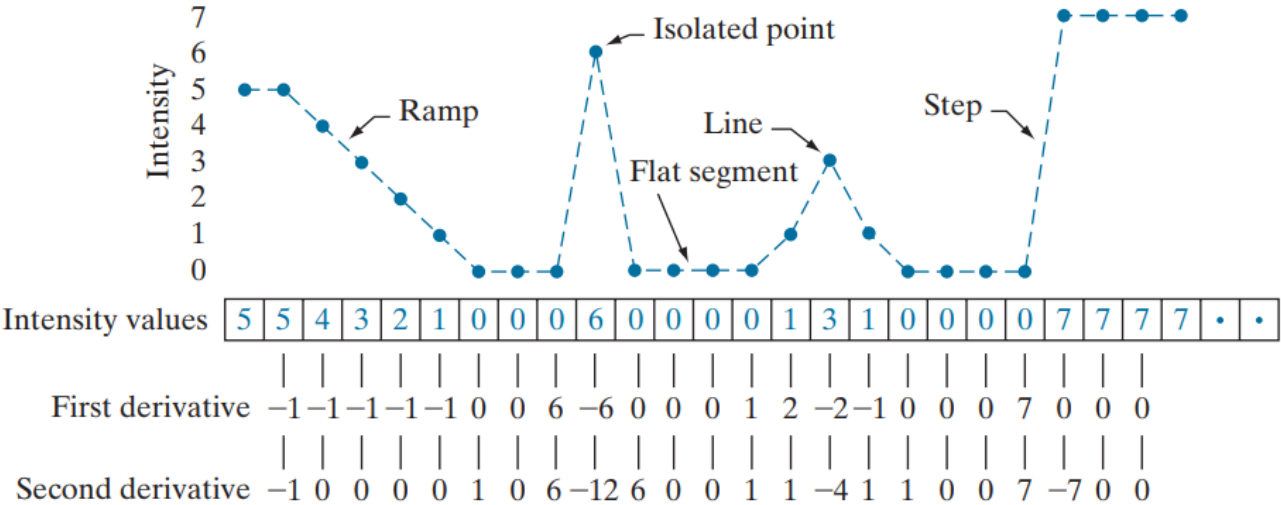
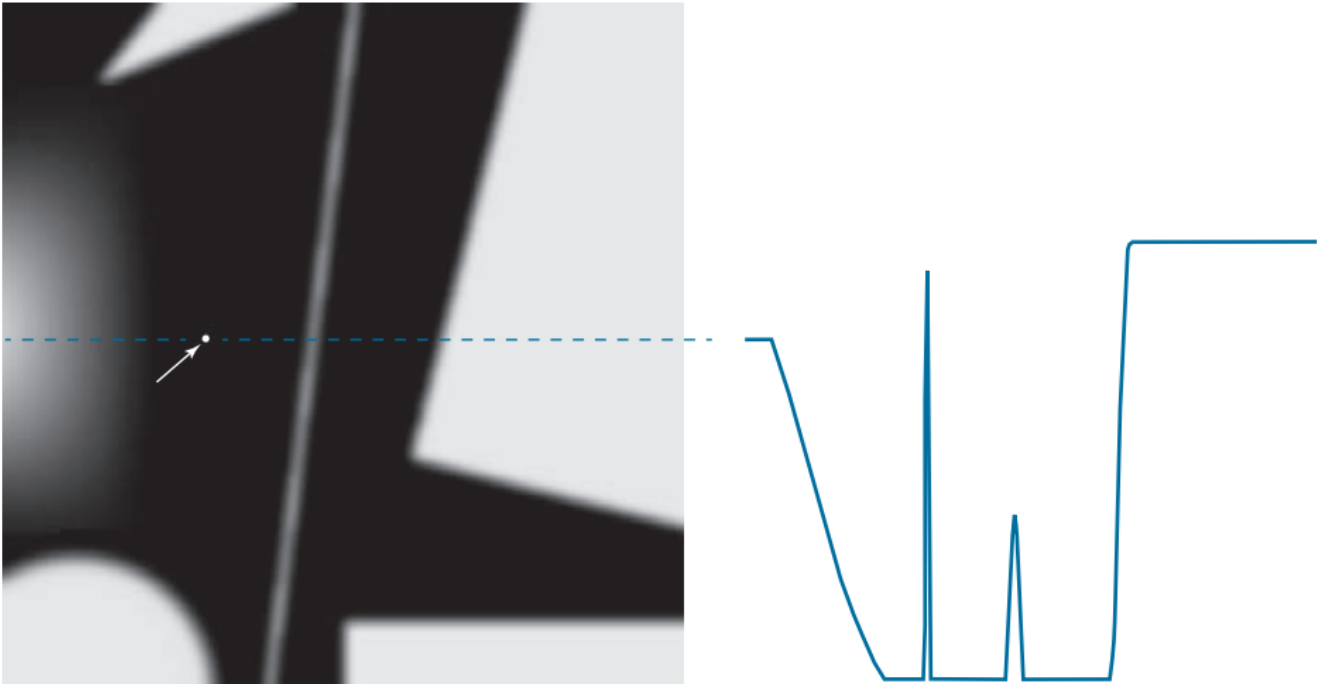
0	1	0	1	1	1
1	-4	1	1	-8	1
0	1	0	1	1	1

➤ Sum of the mask coefficients is 0

- Insight: in areas where the gray values are similar (low-frequency), application of the filter makes the gray values close to zero.

➤ Visual effect: enhances edges and other discontinuities

Example



Derivatives

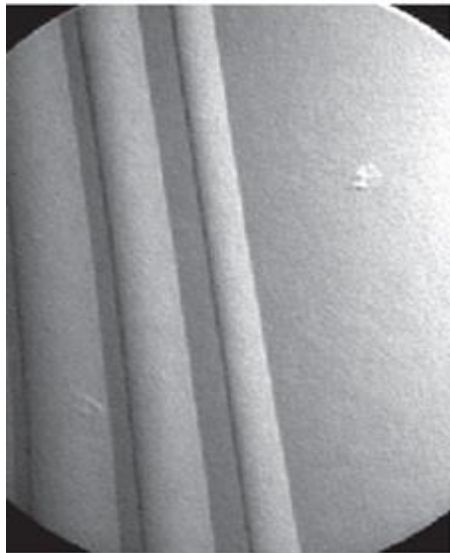
- First-order derivatives generally produce **thicker edges** in an image
- First-order derivatives generally have a stronger response to a intensity-level step, such as **edges**
- Second-order derivatives have a stronger response to fine detail, such as **thin lines, isolated points, and noise**
- Second-order derivatives produce **a double-edge** response at ramp and step transitions in intensity
- The **sign** of the second derivatives can be used to determine whether a transition into an edge is from light to dark or dark to light

Point detection

$$\nabla^2 f(x, y) = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

0	1	0
1	-4	1
0	1	0

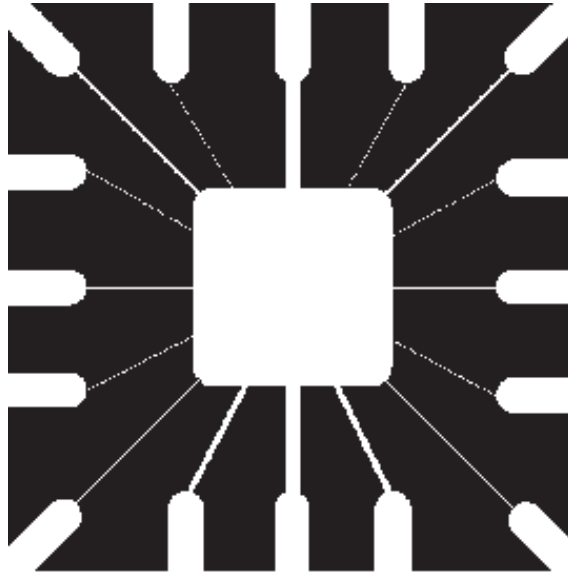
$$\nabla^2 f(x, y) = f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1) - 4f(x, y)$$



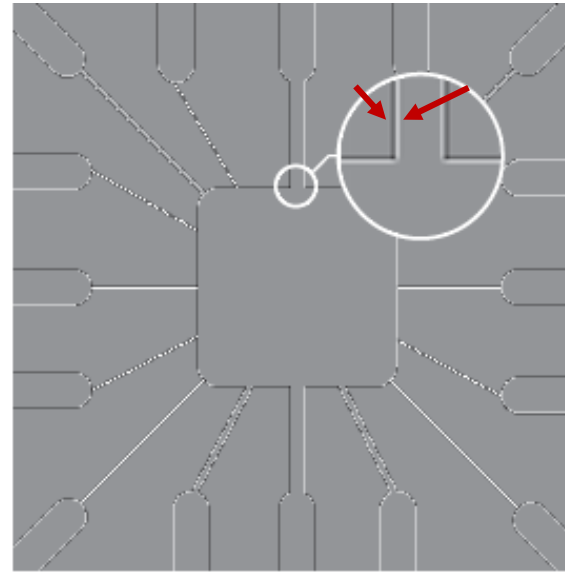
$$|R| \geq T$$

Line detection

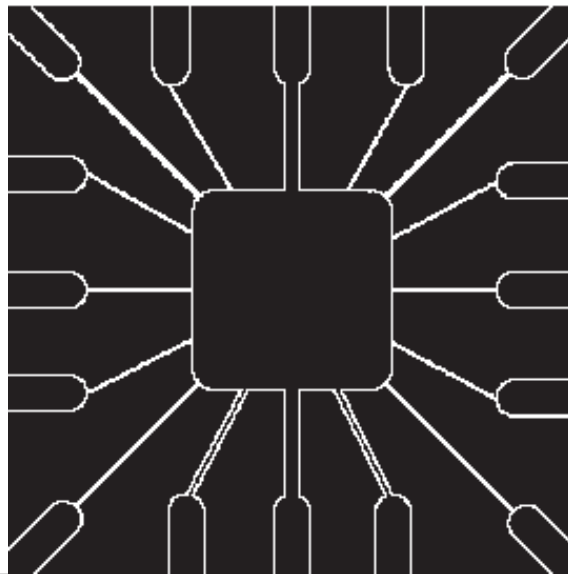
Original



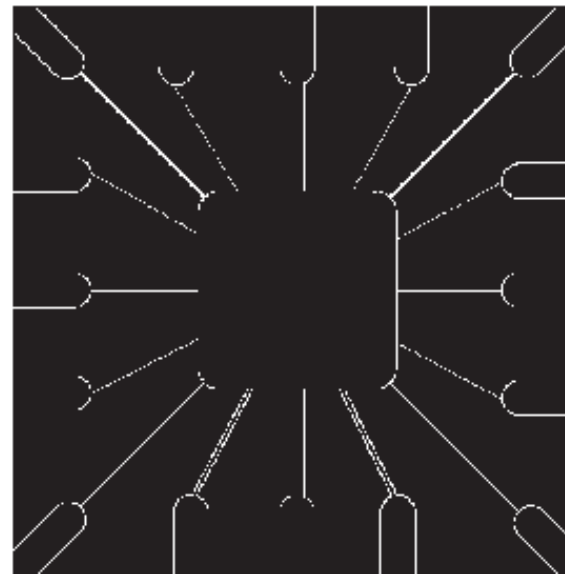
Laplacian
Double edge



Absolute



Positive



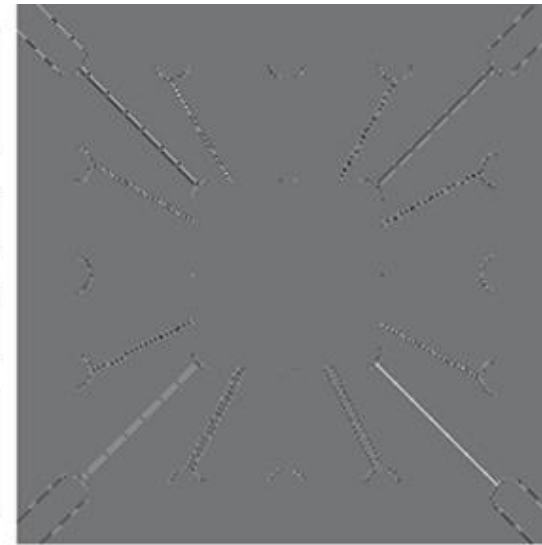
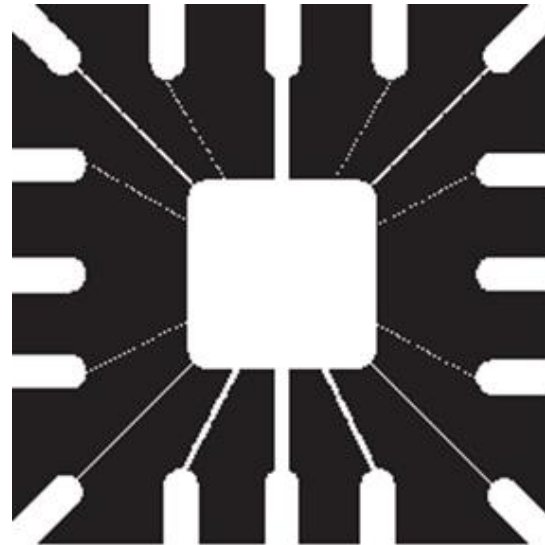
Line detection

- Masks for lines of different directions:

-1	-1	-1	2	-1	-1	-1	2	-1	-1	-1	2
2	2	2	-1	2	-1	-1	2	-1	-1	2	-1
-1	-1	-1	-1	-1	2	-1	2	-1	2	-1	-1
Horizontal			+45°			Vertical			-45°		

- Respond more strongly to lines of one pixel thick of the designated direction.

Line detection



+45° line
detector



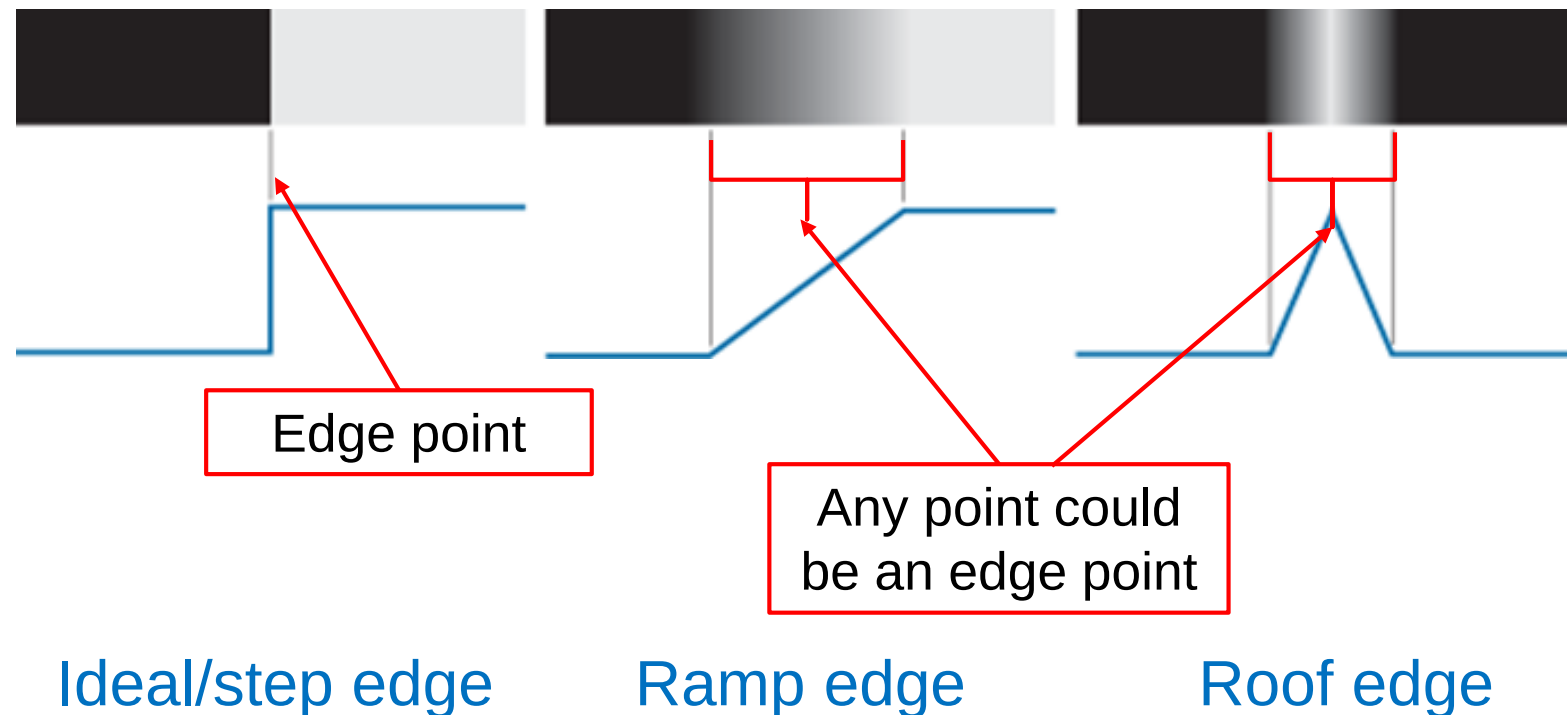
Lower left
corner



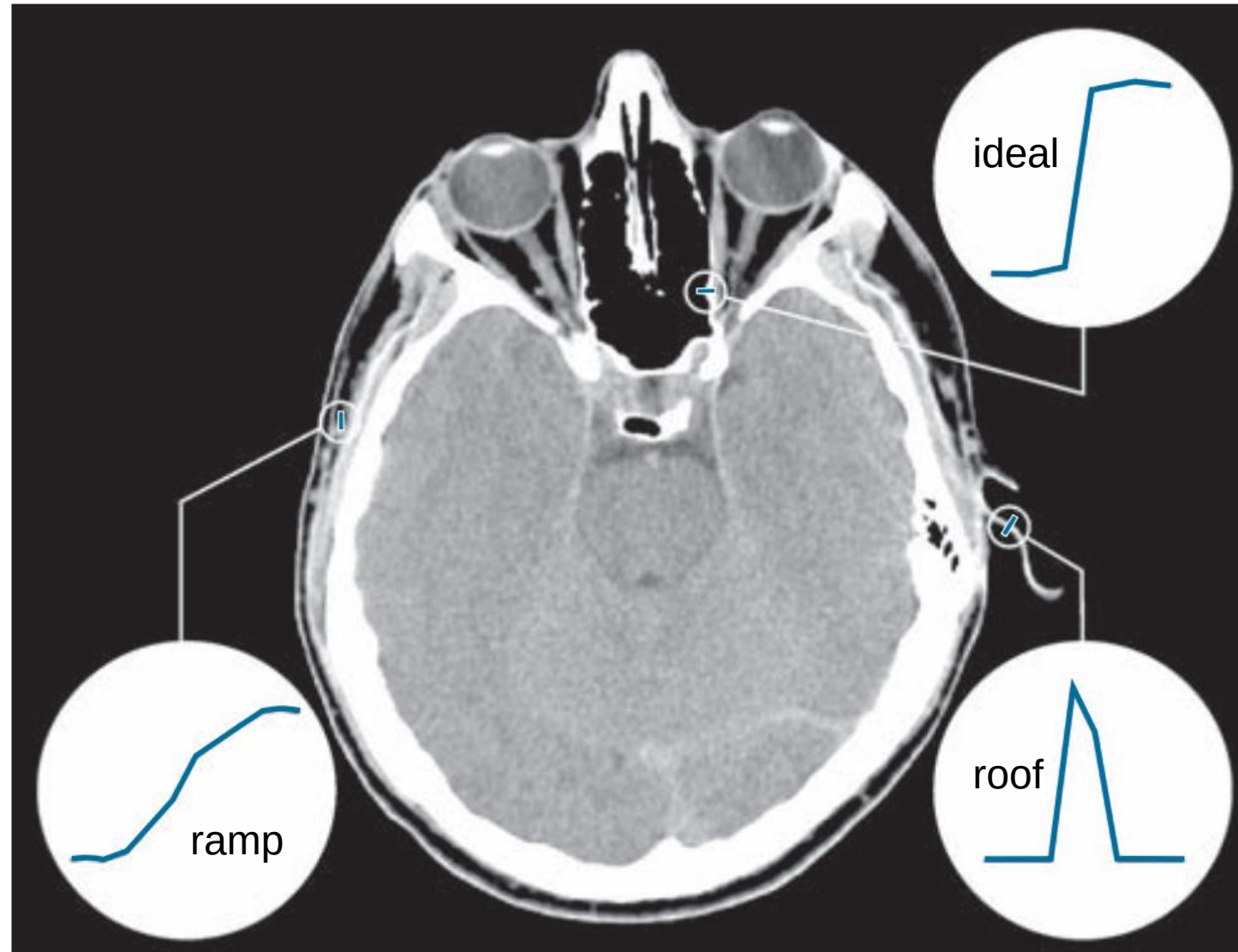
Positive

What is an edge?

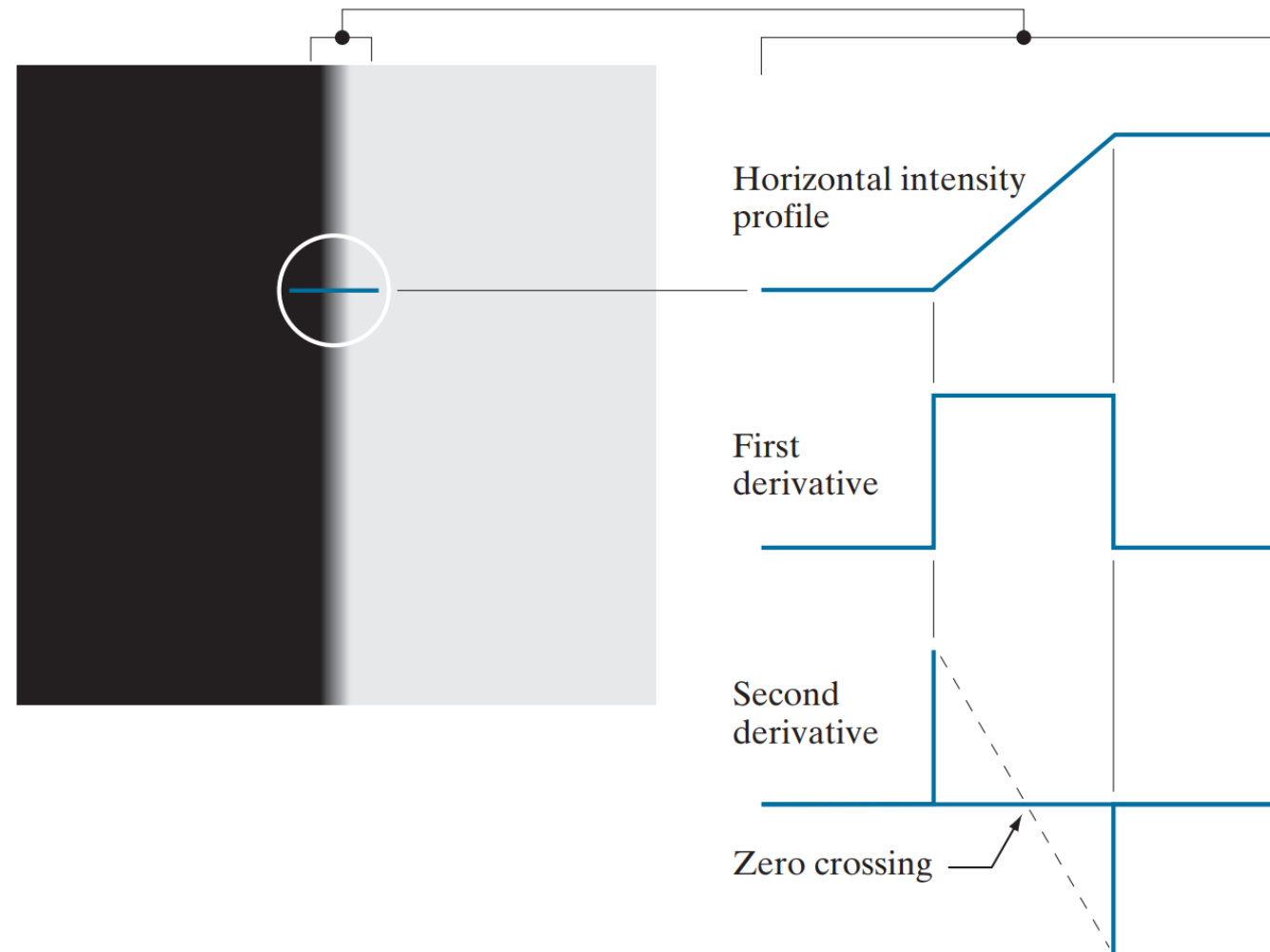
- A set of connected pixels that lie on the boundary between two regions.
- Local concept



What is an edge? – An example



Edge detection – 1st and 2nd order derivatives



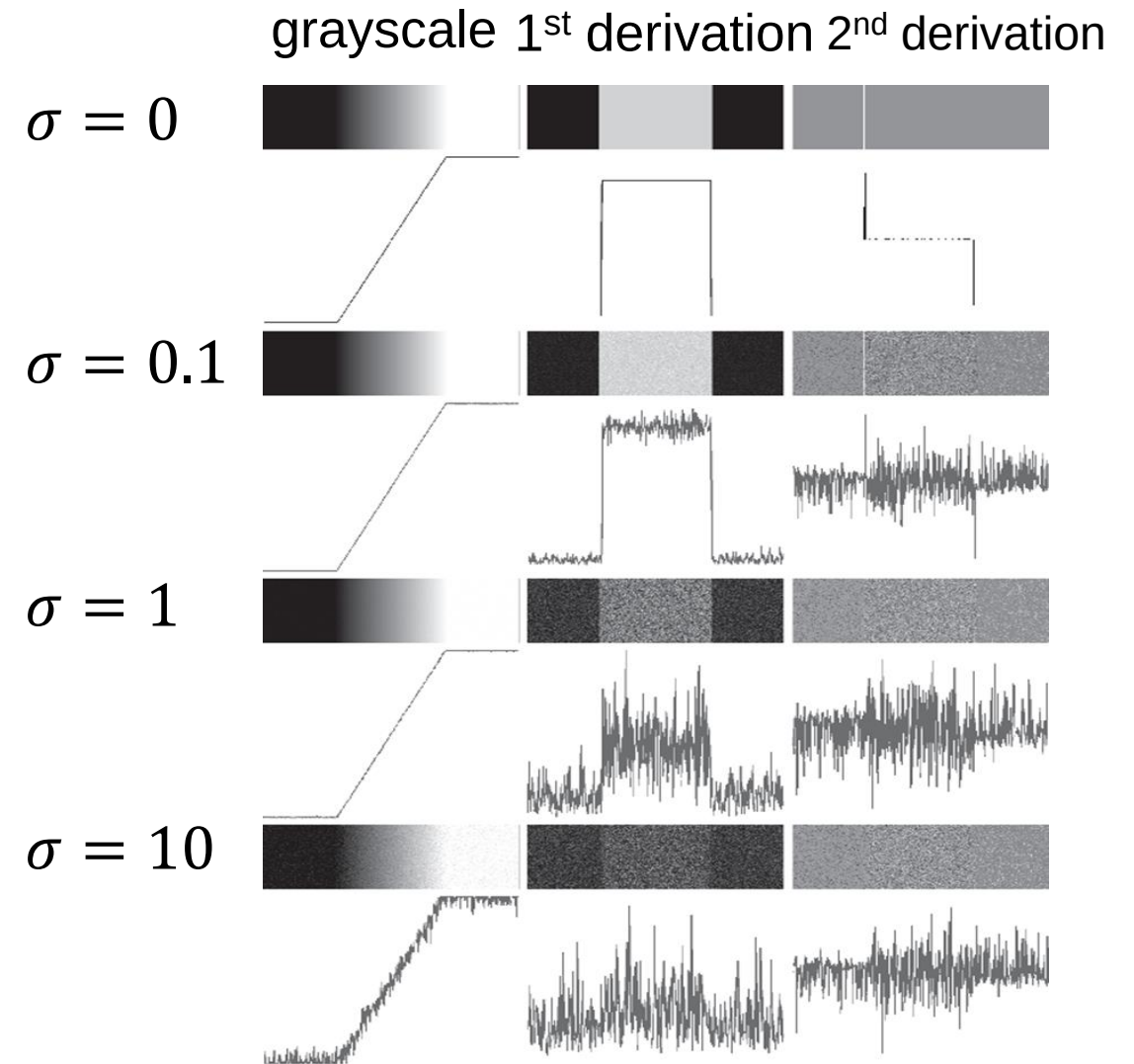
Edge detection – Effect of Noise

- Corrupted by Random Gaussian noise of mean 0 and standard deviation of 0, 0.1, 1, 10.

- Conclusion?

Derivative is sensitive to noise

What to do?



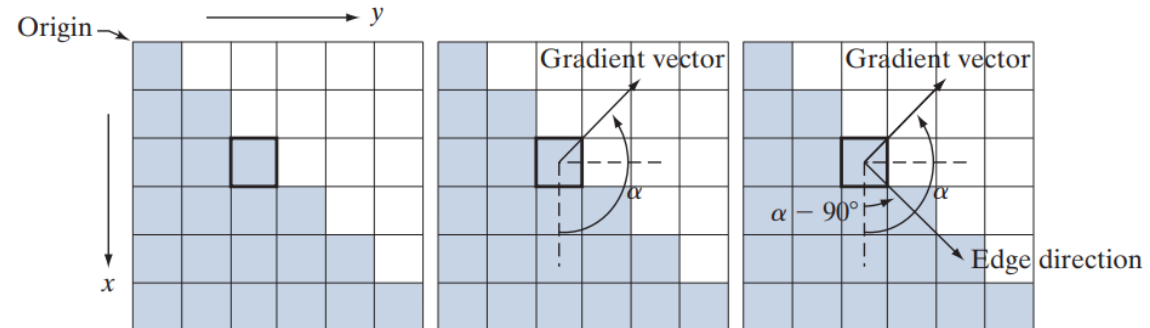
Gradient operators (1st derivative)

- Use gradient for image differentiation
- The gradient of an image $f(x, y)$ at location (x, y) is defined as

$$\nabla f = \text{grad}(f) = \begin{bmatrix} g_x \\ g_y \end{bmatrix} = \begin{bmatrix} \frac{\partial f}{\partial x} & \frac{\partial f}{\partial y} \end{bmatrix}^T$$

- Magnitude $M(x, y) = \text{mag}(\nabla f) = \sqrt{g_x^2 + g_y^2} \approx |g_x| + |g_y|$

- Angle $\alpha(x, y) = \tan^{-1} \left(\frac{g_y}{g_x} \right)$



Finite Gradient -- Approximation

➤ Central differences

$$\Delta_x f(x, y) = \frac{f(x + h, y) - f(x - h, y)}{2h}$$

$$\Delta_y f(x, y) = \frac{f(x, y + h) - f(x, y - h)}{2h}$$

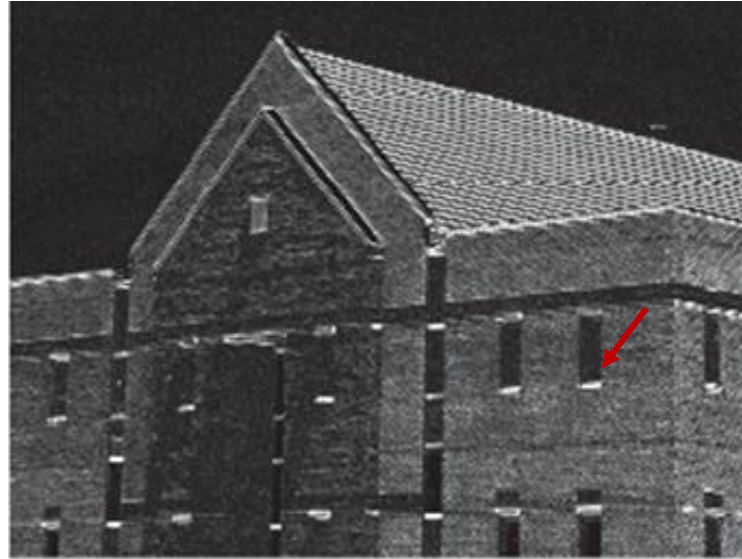
- h is a small integer, usually 1.
- h should be chosen small enough to provide a good approximation to the derivative, but large enough to neglect unimportant changes in the image function.

Edge detection example

Original



$|G_x|$



$|G_y|$



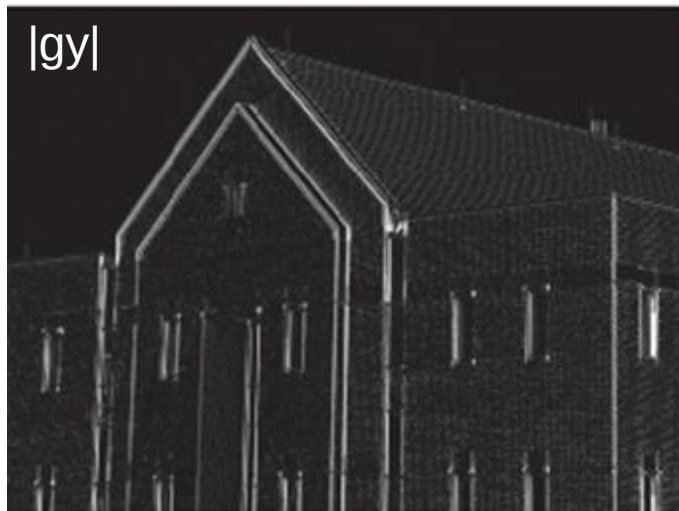
$|G_x| + |G_y|$



Edge detection example

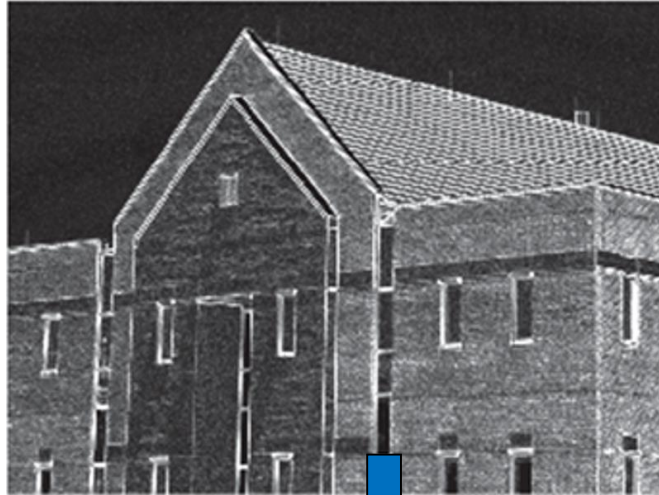


The averaging caused the response of all edges to be weaker.



Edge detection example

Gradient after smoothing



Smooth and
threshold operation:
Highlight the main
edges and maintain a
lot of connectivity.

Thresholding

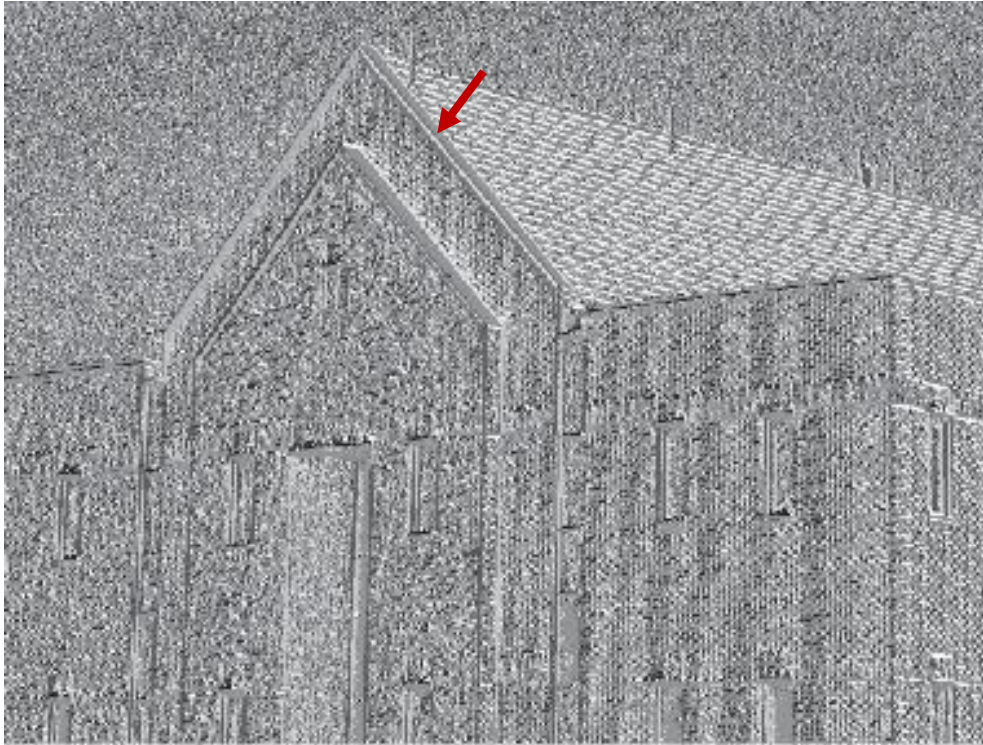


Thresholding



Edge detection example

- Gradient angle



$$\alpha(x, y) = \tan^{-1} \left[\frac{g_y(x, y)}{g_x(x, y)} \right]$$

Areas of constant intensity in this image indicate that the direction of the gradient vector is the same at all the pixel locations in those regions

Edge operator

$$g_x(x, y) = \frac{\partial f(x, y)}{\partial x} = f(x + 1, y) - f(x, y)$$

$$g_y(x, y) = \frac{\partial f(x, y)}{\partial y} = f(x, y + 1) - f(x, y)$$

Diagonal edges

$$G_x = (z_9 - z_5)$$

$$G_y = (z_8 - z_6)$$

Middle

$$G_x = (z_7 + z_8 + z_9) - (z_1 + z_2 + z_3)$$

$$G_y = (z_3 + z_6 + z_9) - (z_1 + z_4 + z_7)$$

Smoothing

$$G_x = (z_7 + 2z_8 + z_9) - (z_1 + 2z_2 + z_3)$$

$$G_y = (z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)$$

-1
1

-1	1
----	---

-1	0	0	-1
0	1	1	0

Roberts

-1	-1	-1	-1	0	1
0	0	0	-1	0	1
1	1	1	-1	0	1

Prewitt

-1	-2	-1	-1	0	1
0	0	0	-2	0	2
1	2	1	-1	0	1

Sobel

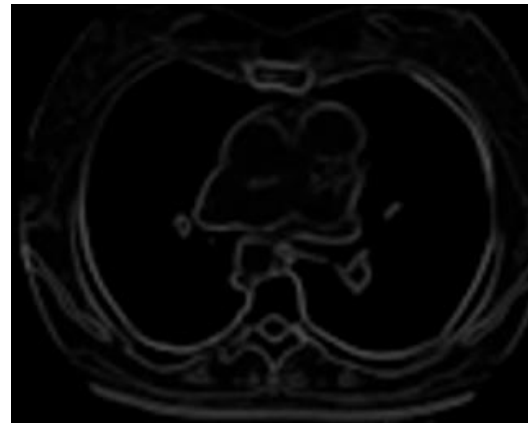
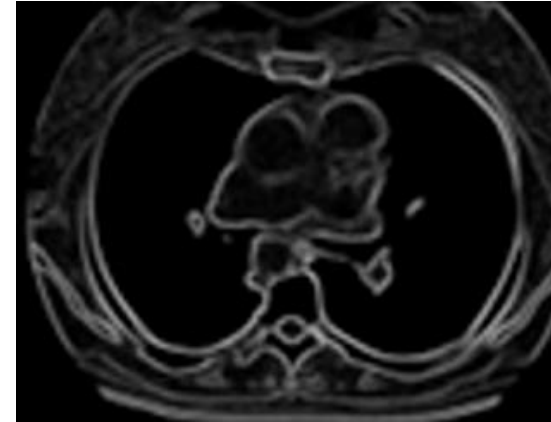
$$\nabla f \approx |G_x| + |G_y|$$

Edge detection example

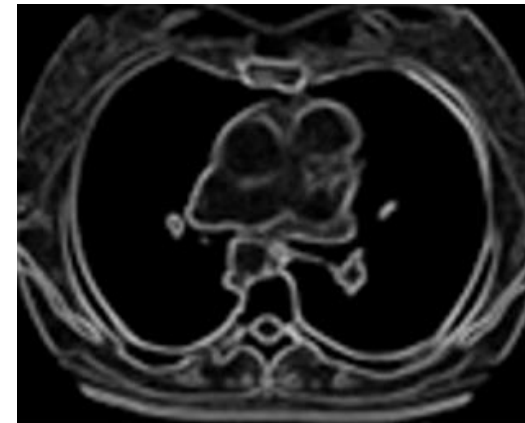
Original



Prewitt



Robert



Sobel

Edge operator

Kirsch
compass
kernels

<table><tr><td>-3</td><td>-3</td><td>5</td></tr><tr><td>-3</td><td>0</td><td>5</td></tr><tr><td>-3</td><td>-3</td><td>5</td></tr></table> N	-3	-3	5	-3	0	5	-3	-3	5	<table><tr><td>-3</td><td>5</td><td>5</td></tr><tr><td>-3</td><td>0</td><td>5</td></tr><tr><td>-3</td><td>-3</td><td>-3</td></tr></table> NW	-3	5	5	-3	0	5	-3	-3	-3	<table><tr><td>5</td><td>5</td><td>5</td></tr><tr><td>-3</td><td>0</td><td>-3</td></tr><tr><td>-3</td><td>-3</td><td>-3</td></tr></table> W	5	5	5	-3	0	-3	-3	-3	-3	<table><tr><td>5</td><td>5</td><td>-3</td></tr><tr><td>5</td><td>0</td><td>-3</td></tr><tr><td>-3</td><td>-3</td><td>-3</td></tr></table> SW	5	5	-3	5	0	-3	-3	-3	-3
-3	-3	5																																					
-3	0	5																																					
-3	-3	5																																					
-3	5	5																																					
-3	0	5																																					
-3	-3	-3																																					
5	5	5																																					
-3	0	-3																																					
-3	-3	-3																																					
5	5	-3																																					
5	0	-3																																					
-3	-3	-3																																					
<table><tr><td>5</td><td>-3</td><td>-3</td></tr><tr><td>5</td><td>0</td><td>-3</td></tr><tr><td>5</td><td>-3</td><td>-3</td></tr></table> S	5	-3	-3	5	0	-3	5	-3	-3	<table><tr><td>-3</td><td>-3</td><td>-3</td></tr><tr><td>5</td><td>0</td><td>-3</td></tr><tr><td>5</td><td>5</td><td>-3</td></tr></table> SE	-3	-3	-3	5	0	-3	5	5	-3	<table><tr><td>-3</td><td>-3</td><td>-3</td></tr><tr><td>-3</td><td>0</td><td>-3</td></tr><tr><td>5</td><td>5</td><td>5</td></tr></table> E	-3	-3	-3	-3	0	-3	5	5	5	<table><tr><td>-3</td><td>-3</td><td>-3</td></tr><tr><td>-3</td><td>0</td><td>5</td></tr><tr><td>-3</td><td>5</td><td>5</td></tr></table> NE	-3	-3	-3	-3	0	5	-3	5	5
5	-3	-3																																					
5	0	-3																																					
5	-3	-3																																					
-3	-3	-3																																					
5	0	-3																																					
5	5	-3																																					
-3	-3	-3																																					
-3	0	-3																																					
5	5	5																																					
-3	-3	-3																																					
-3	0	5																																					
-3	5	5																																					

Line
detector

-1	-1	-1	2	-1	-1	-1	2	-1	-1	-1	2
2	2	2	-1	2	-1	-1	2	-1	-1	2	-1
-1	-1	-1	-1	-1	2	-1	2	-1	2	-1	-1
Horizontal			+45°			Vertical			-45°		

Edge detection example

Kirsch kernel



Both masks have similar response to horizontal and vertical edges, but is weaker than that by Sobel masks.

5	5	5
-3	0	-3
-3	-3	-3

W

5	5	-3
5	0	-3
-3	-3	-3

SW

2nd Derivative: Laplacian Operator

- The Laplacian operator (∇^2) is an operator approximating the second derivative which gives the differentiation magnitude.

We discussed this operator in spatial filtering $\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$

- It is isotropic

0	-1	0
-1	4	-1
0	-1	0

4-neighborhood

-1	-1	-1
-1	8	-1
-1	-1	-1

8-neighborhood

Issues with Laplacian

➤ Problems:

- Unacceptably sensitive to noise
- Magnitude of Laplacian results in double edges
- Does not provide gradient, so can't detect edge direction

➤ Fixes:

- Smoothing → LoG
- Using zero-crossing property for edge location
- Not for gradient direction, but for establishing whether a pixel is on the dark or light side of an edge

Laplacian of a Gaussian (LoG)

- Consider a Gaussian function:

$$h(r) = e^{-\frac{r^2}{2\sigma^2}}$$

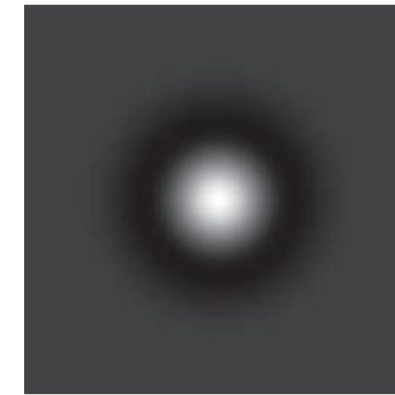
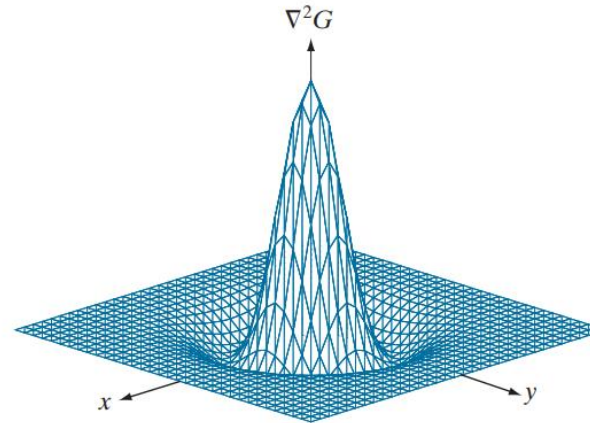
- where $r^2 = x^2 + y^2$, σ is the standard deviation. Convoluting this function with an image blurs the image.
- The Laplacian of a Gaussian (LoG) is:

$$\nabla^2 h(r) = \left[\frac{r^2 - 2\sigma^2}{\sigma^4} \right] e^{-\frac{r^2}{2\sigma^2}}$$

- Convoluting an image with LoG is the same as convoluting the image with the Gaussian smoothing function h first and then computing the Laplacian of the result.

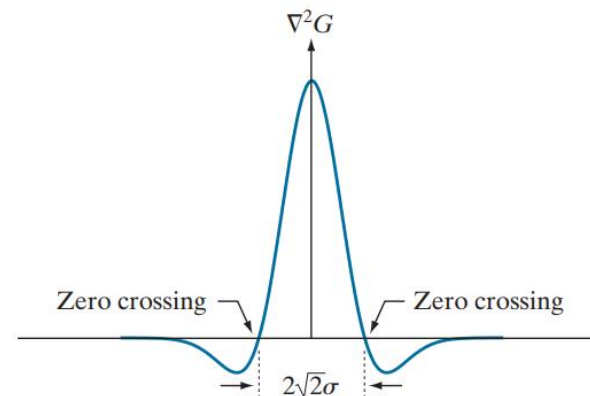
The LoG (cont')

$$\nabla^2 h(r) = \left[\frac{r^2 - 2\sigma^2}{\sigma^4} \right] e^{-\frac{r^2}{2\sigma^2}}$$



$$g(x, y) = [\nabla^2 G(x, y)] * f(x, y)$$

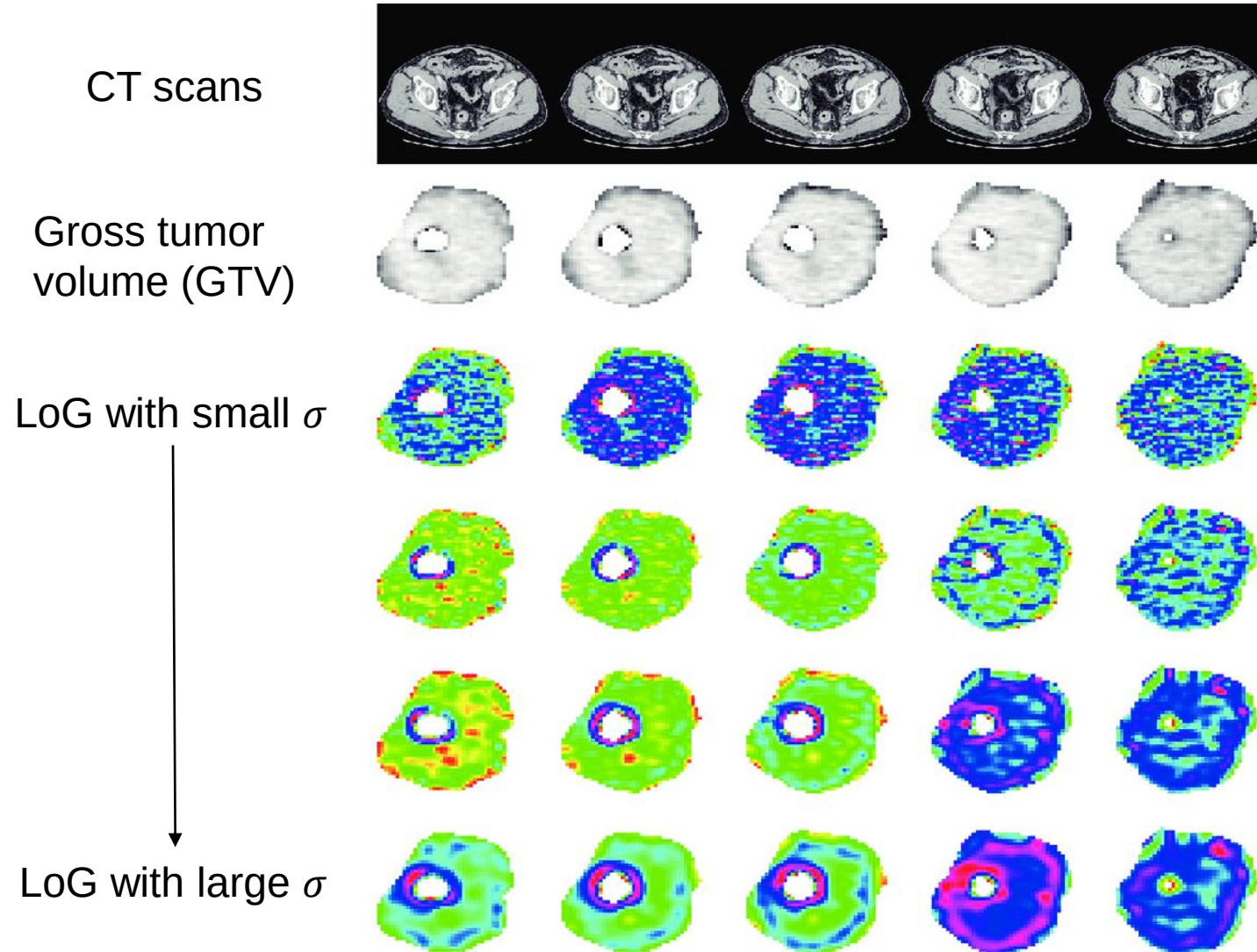
$$g(x, y) = \nabla^2 [G(x, y) * f(x, y)]$$



0	0	-1	0	0
0	-1	-2	-1	0
-1	-2	16	-2	-1
0	-1	-2	-1	0
0	0	-1	0	0

- LoG sometimes is called the **Mexican hat function**.
- Certain aspects of human vision can be modeled by this function.

The LoG (cont')



Edge detection: high-level algorithm

- Active contour model, also called "snakes"

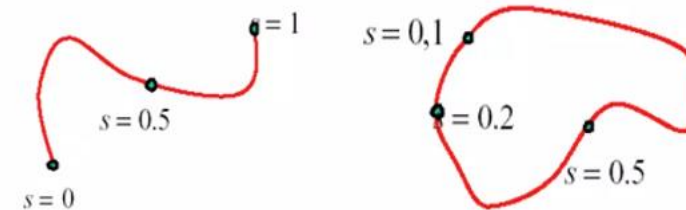


1987, by Michael Kass, Andrew Witkin, Demetri Terzopoulos

Active contour model

- The contours defined in the (x, y) plane of an image as a parametric curve

$$\mathbf{v}(s) = (x(s), y(s)), 0 \leq s \leq 1$$



- Contours said to possess an energy (E_{snake}) which is defined as the sum of the three energy terms.

$$E_{snake} = E_{internal} + E_{external} + E_{constraint}$$

- The energy terms are defined cleverly in a way such that the final position of the contour will have a minimum energy (E_{min})
- Therefore our problem of detecting objects reduces to an energy minimization problem.

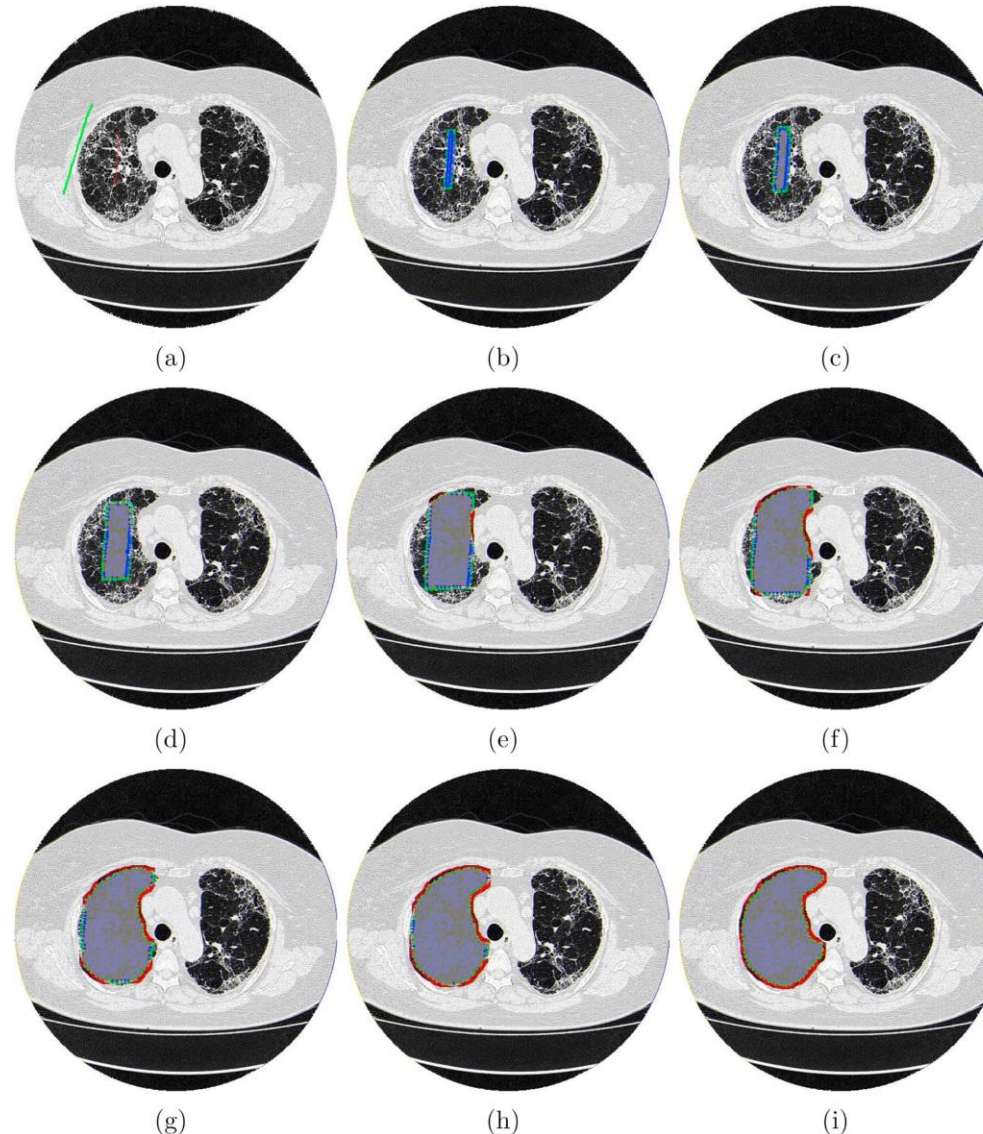
$$E_{total} = \int_s \left(\alpha \left| \frac{\partial}{\partial s} \bar{v} \right|^2 + \beta \left| \frac{\partial^2}{\partial s^2} \bar{v} \right|^2 + E_{ext}(\bar{v}(s)) \right) ds$$

Active contour model

- Lung segmentation

Steps:

- (a) Markings to generate the knowledge base;
- (b) Initialization;
- (c)–(h) Evolution of the curve;
- (i) Segmentation result



Thresholding

$$g(x, y) = \begin{cases} 1 & \text{if } f(x, y) > T \\ 0 & \text{if } f(x, y) \leq T \end{cases}$$

- Any point (x, y) for which $f(x, y) > T$ is called an object point; otherwise, the point is called a background point.

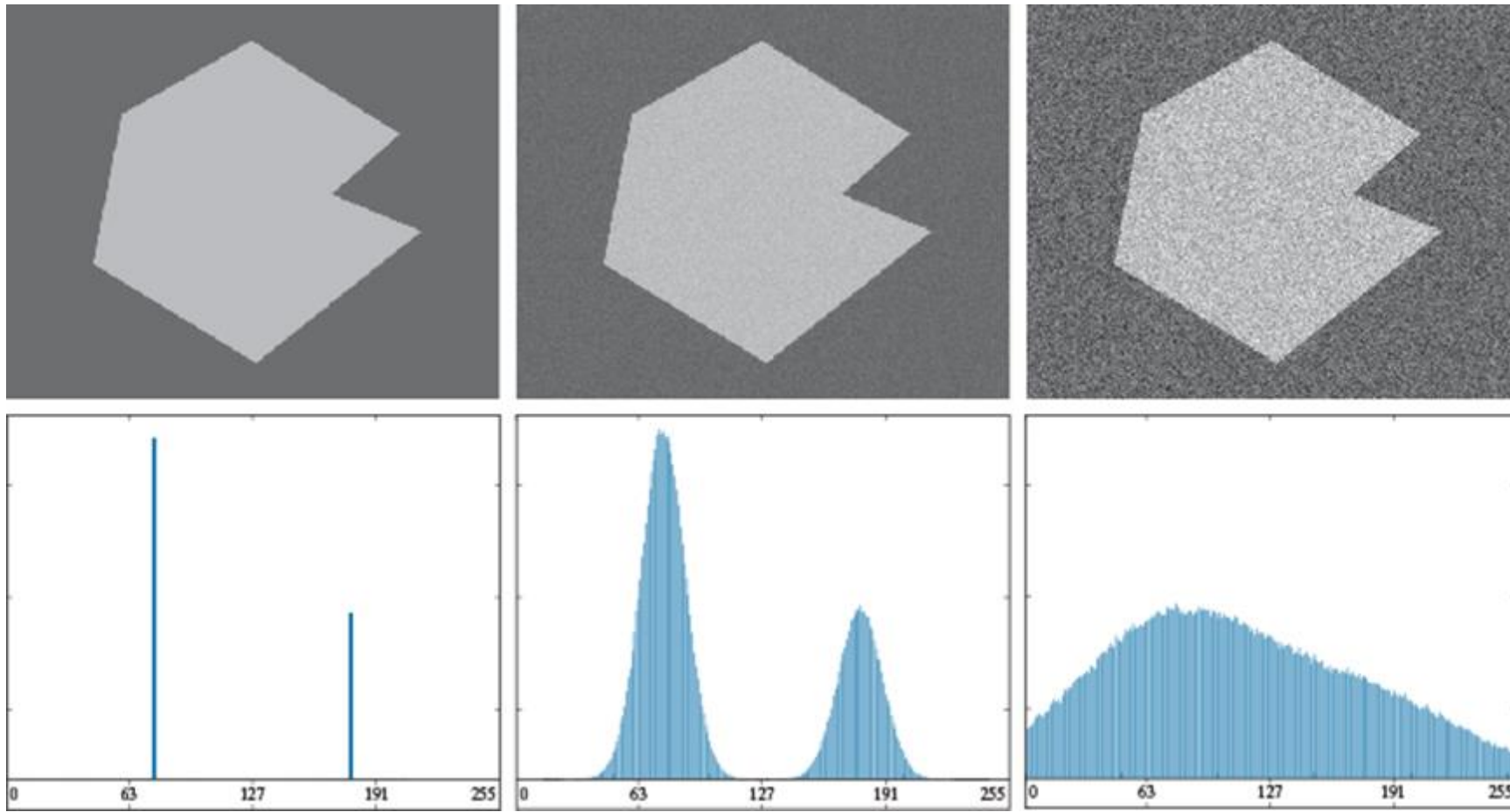
$$G = T[x, y, p(x, y), f(x, y)]$$

Some local property

Grayscale value

- When T depends only on $f(x, y)$, the threshold is called **global**.
- If T depends on both $f(x, y)$ and $p(x, y)$, the threshold is **local**.
- If T depends on the spatial coordinates x & y , the threshold is called **dynamic or adaptive**.

The role of noise in image thresholding



The role of illumination & reflectance

Non-uniform illumination Compensation

- Image formation model

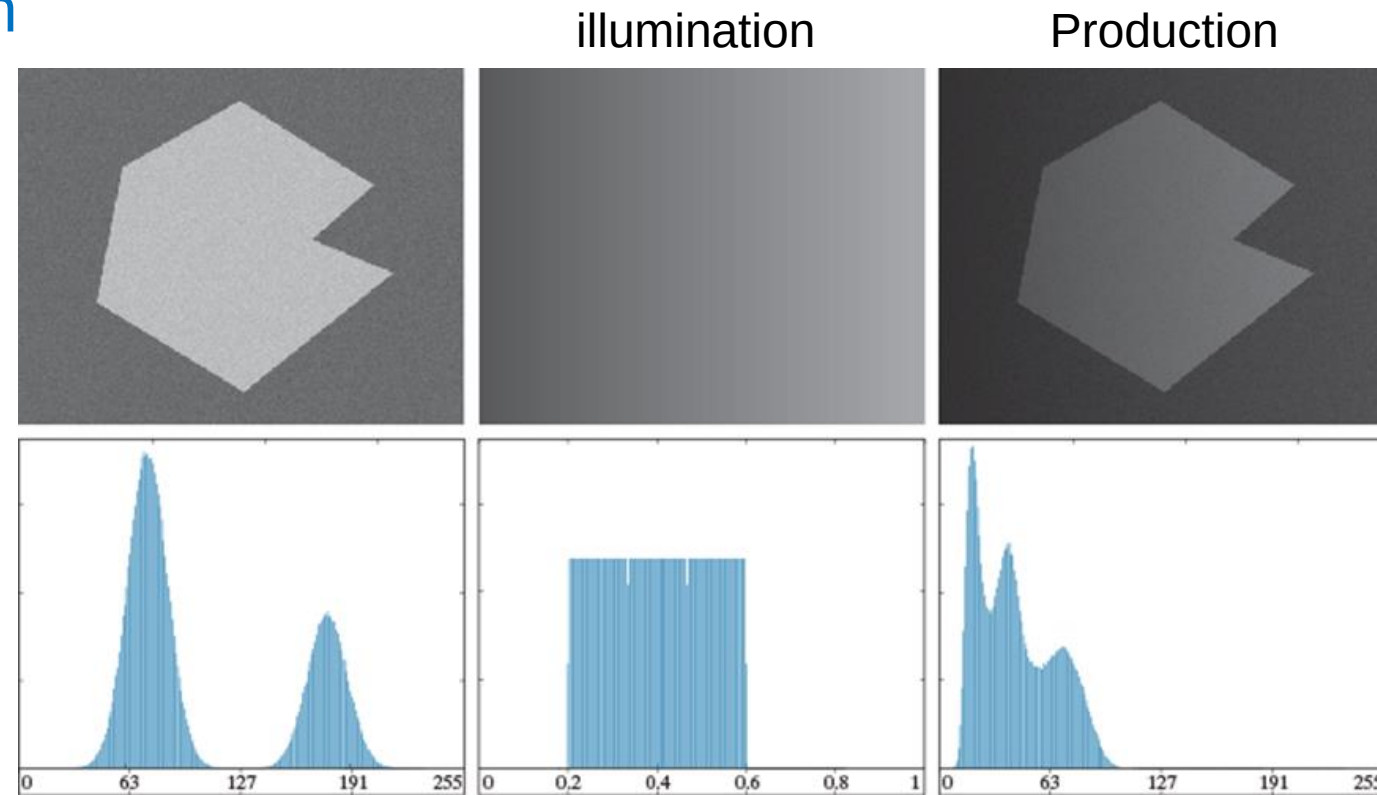
$$f(x, y) = i(x, y)r(x, y)$$

- First take a shot of a constant, white reflective surface

$$g(x, y) = ki(x, y)$$

- For any image $f(x, y)$, segment the normalized image $h(x, y)$

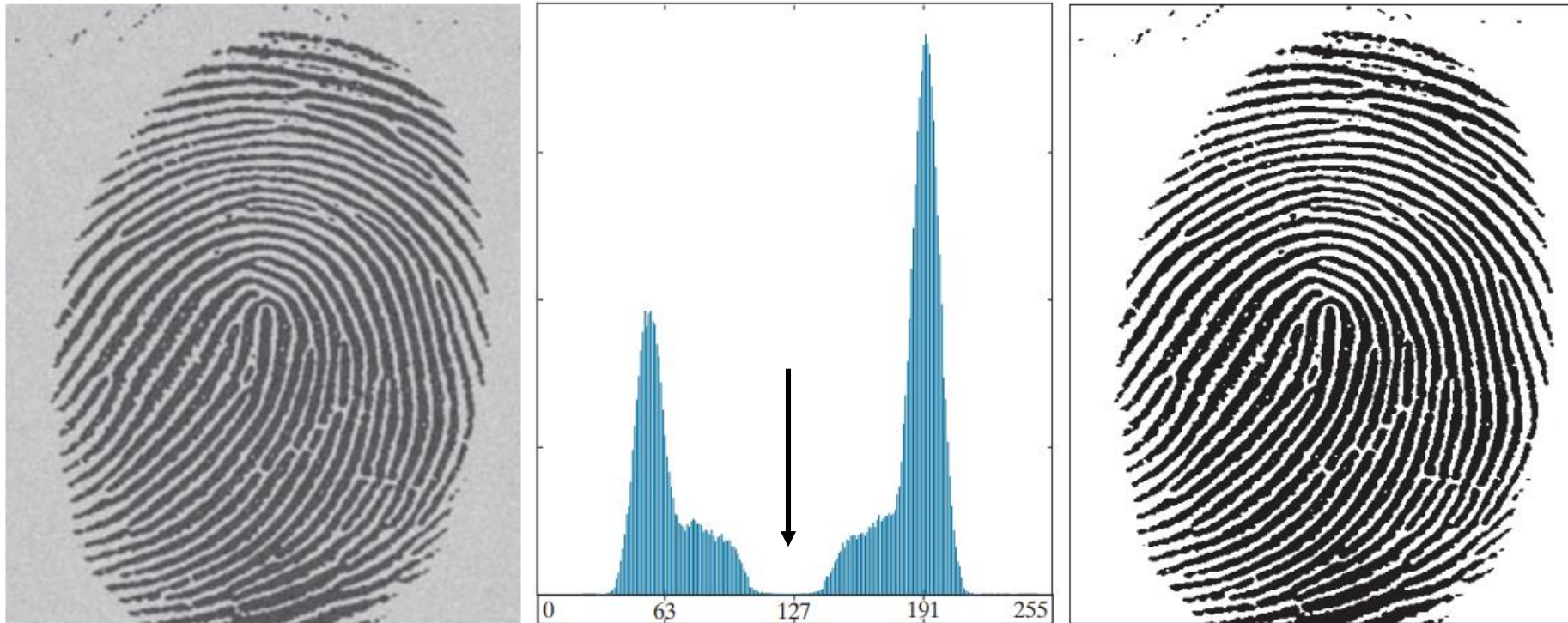
$$h(x, y) = f(x, y)/g(x, y) = r(x, y)/k$$



Basic Global Thresholding

1. Select an initial estimate for T
 - If foreground and background occupy comparable areas, average gray-level might be good.
 - If the areas not comparable, midway between max and min gray-level might be better.
2. Segment the image using T to generate 2 regions, G_1 & G_2 .
3. Compute the average gray level values m_1 and m_2 for regions G_1 and G_2 , respectively.
4. Compute the new T as $(m_1 + m_2)/2$.
5. Repeat the steps 2 ~ 4 until the T converges.

Example



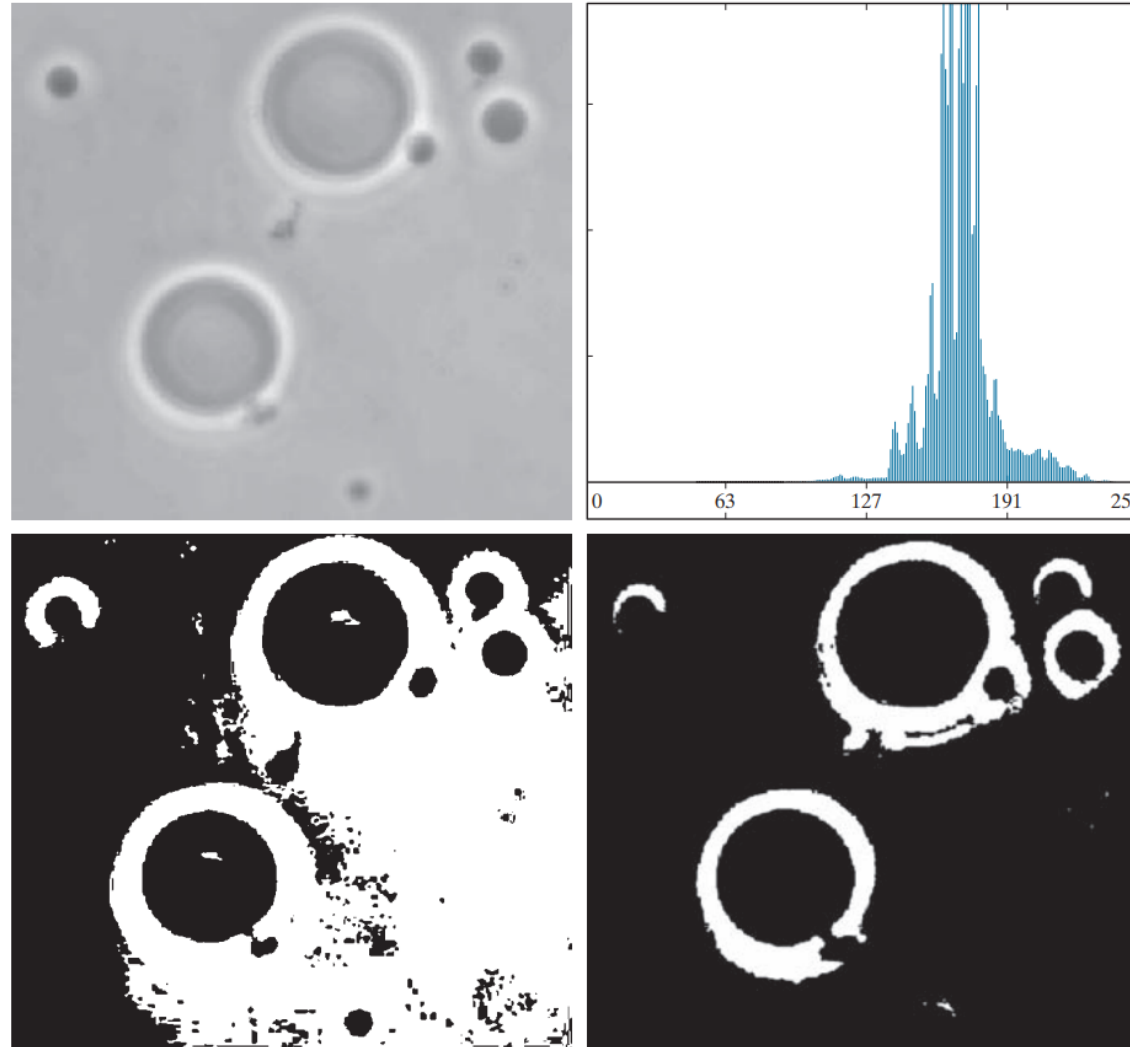
Initial T is the average gray-level. The final T is 125.4 with 3 iterations.

Otsu's Method

- A popular method for optimum global thresholding
- Obtain the image histogram $p_i = n_i/N$
- The image average is $m_G = \sum_{i=0}^{L-1} ip_i$
- Mean intensity up to image level k : $m(k) = \sum_{i=0}^k ip_i$
- If we threshold at level k , we get $\omega(k) = \sum_{i=0}^k p_i$, $\mu(k) = \sum_{i=k+1}^{L-1} p_i$
- We would like to maximize the inter-class variance
- k is set to the value that maximize $\sigma_B^2(k) = \frac{[m_G\omega(k)-m(k)]^2}{\omega(k)\mu(k)}$

Inter-class variance

Otsu's Method -- Example

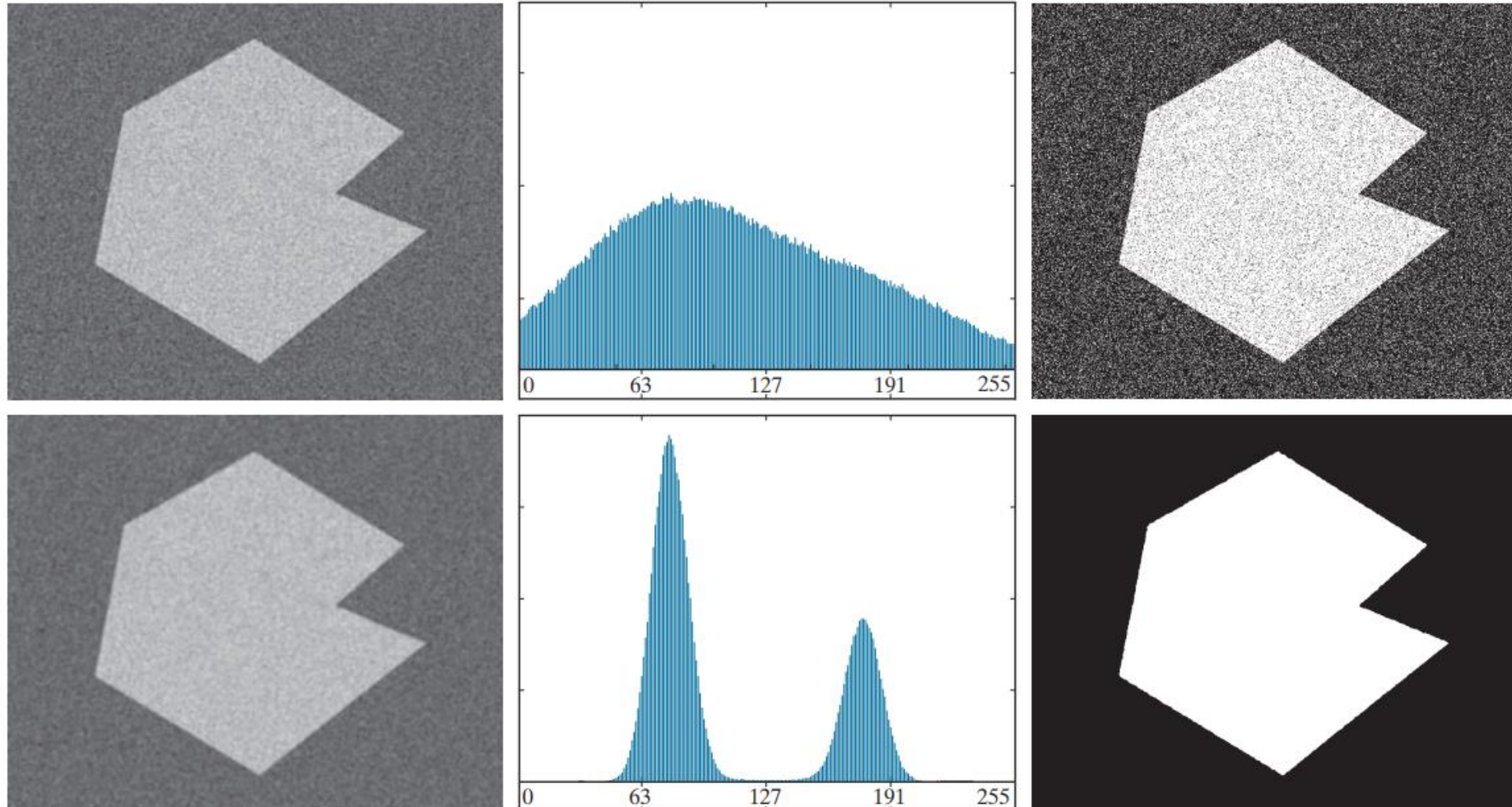


Basic global

Otsu

Using smoothing to improve global thresholding

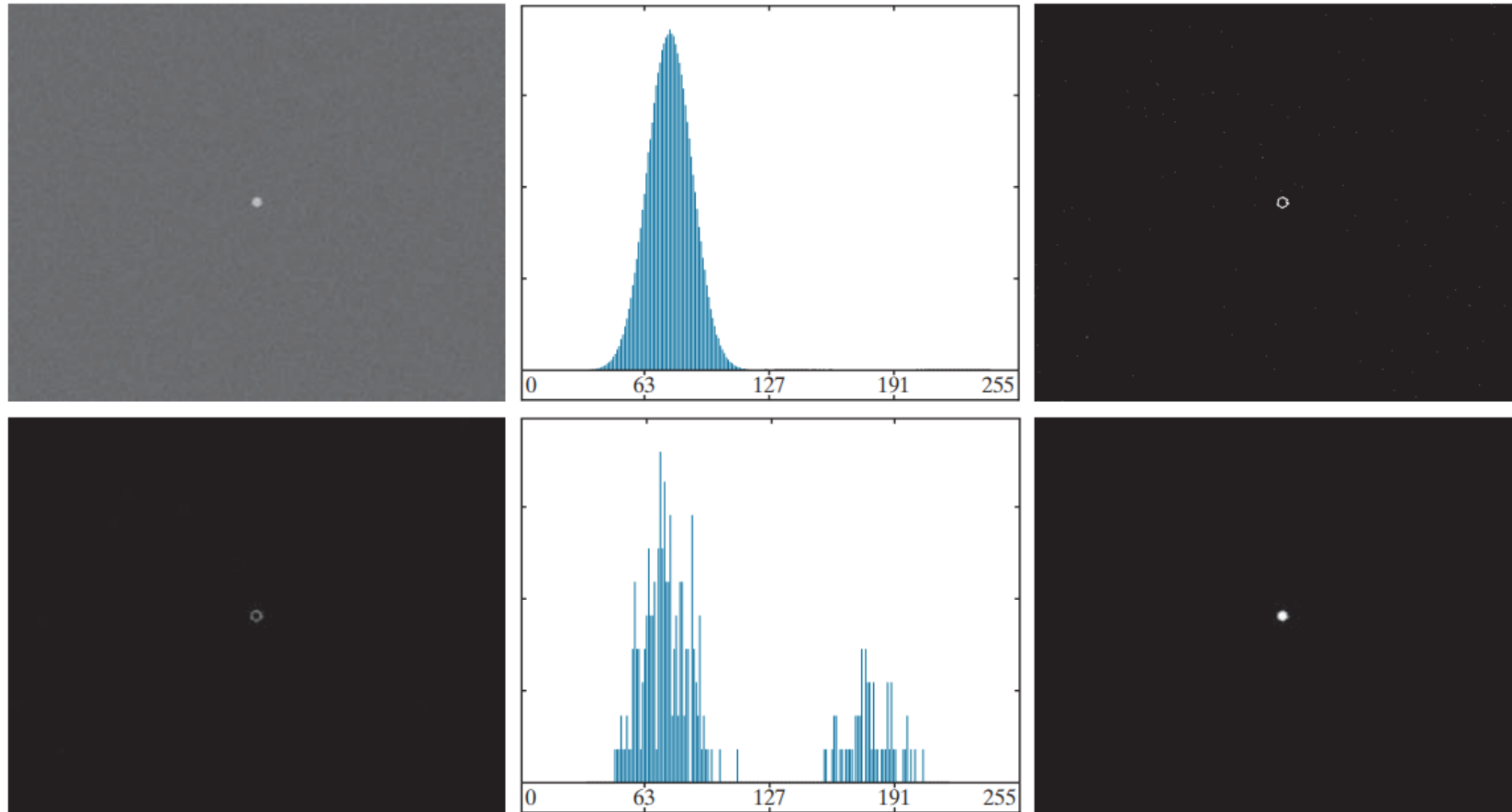
Otsu



Use of Edge Characteristics

- It is easier to select T if the histogram peaks are tall, narrow, symmetric and separated by deep valley.
- If relying on only gray-level information, the result is susceptible to relative sizes of the object and background.
- One possible solution: using only pixels on or near the edges.

Using edges to improve global thresholding

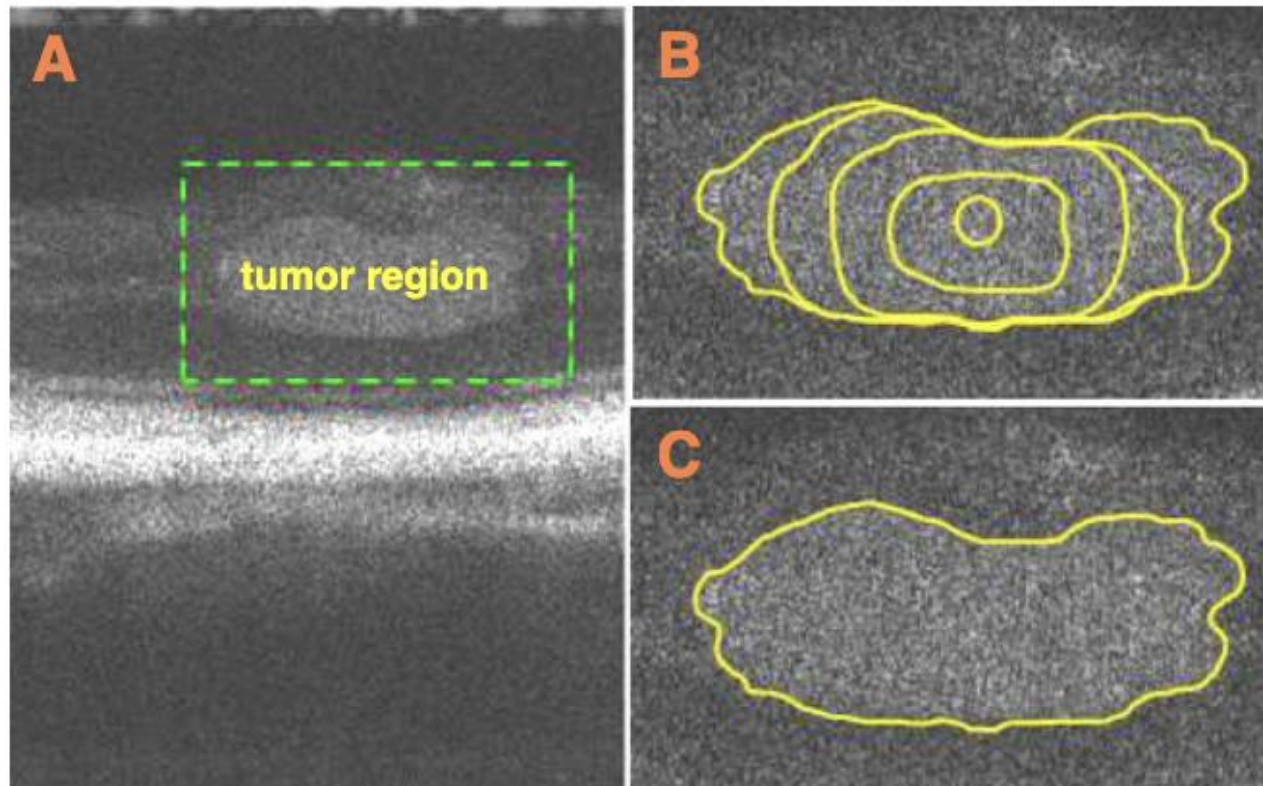


a	b	c
d	e	f

(a) Noisy image from Fig. 10.38(a) and (b) its histogram. (c) Mask image formed as the gradient magnitude image thresholded at the 99.7 percentile. (d) Image formed as the product of (a) and (c). (e) Histogram of the nonzero pixels in the image in (d). (f) Result of segmenting image (a) with the Otsu threshold based on the histogram in (e).

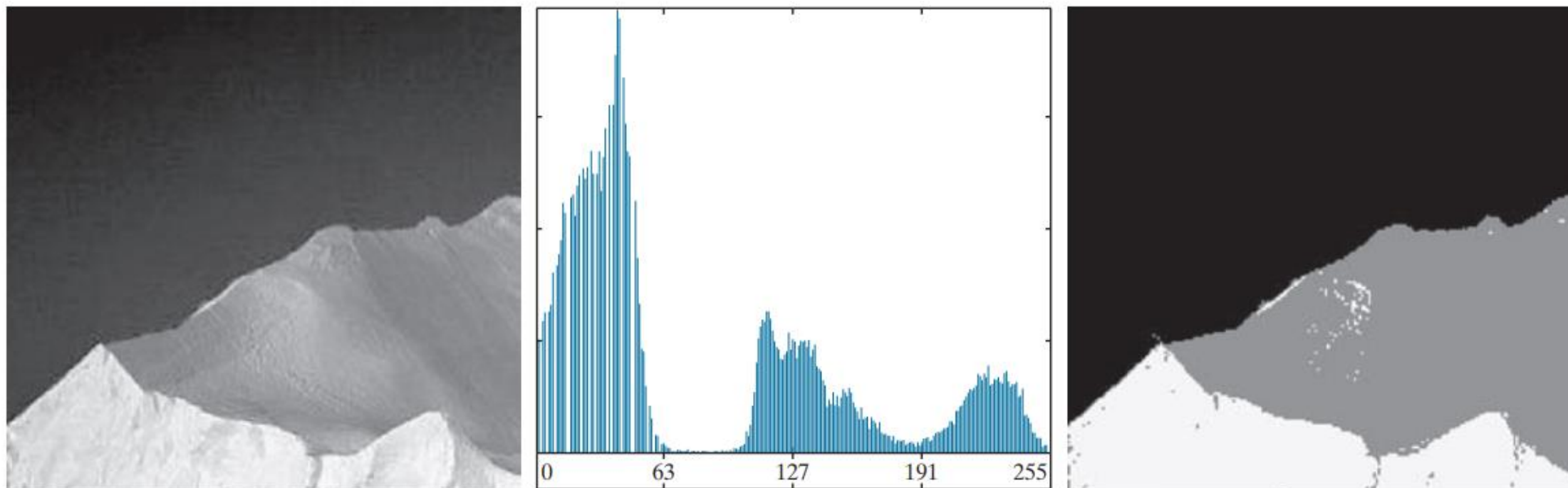
Use of edges characteristics

A tumor in a mouse retina(OCT image)



Segmentation through
edge features

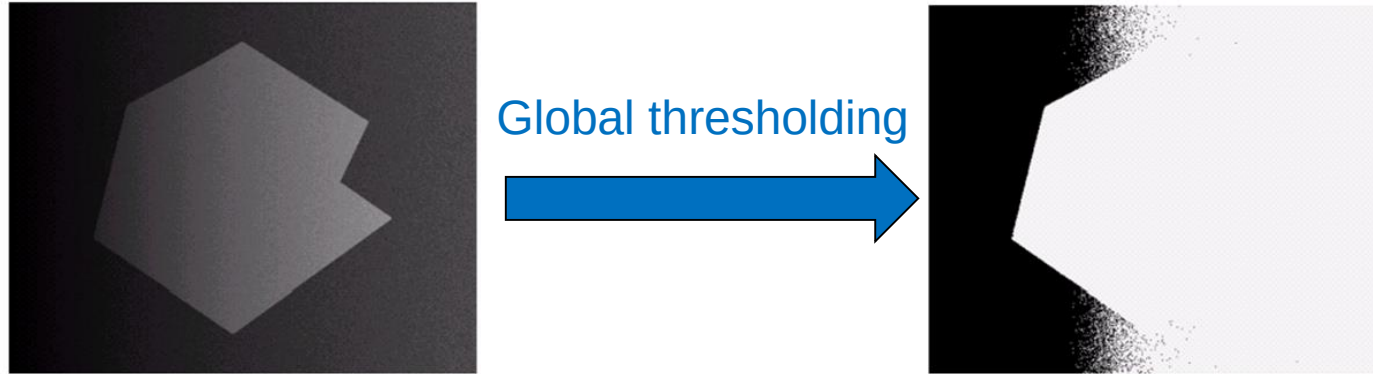
Multiple Thresholds



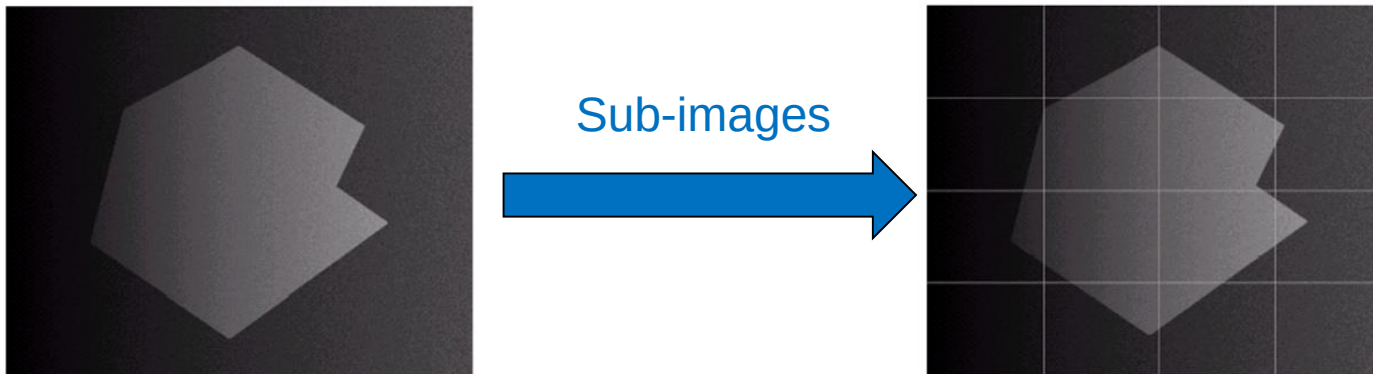
Dual threshold

Basic Adaptive Thresholding

- Global thresholding often fails in the case of uneven illumination.



- The simplest fix is to
- Divide the image into sub-images
 - Determine T for each sub-image

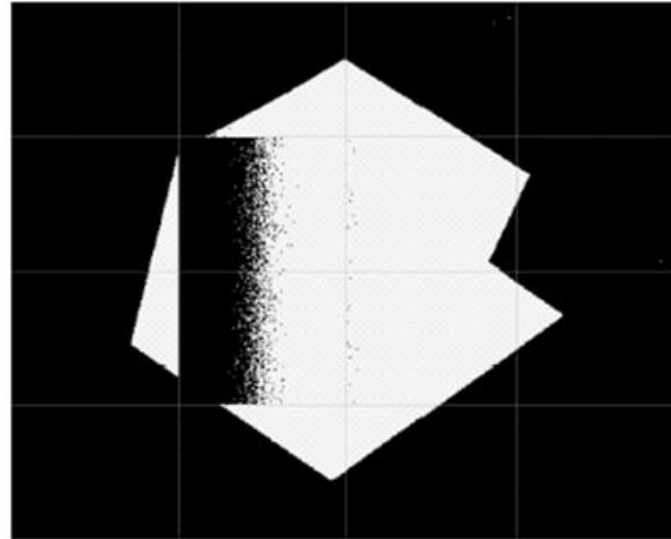


How to deal with the Sub-images

➤ Observation:

- Type 1: For all sub-images, if the gray-level variance is $< T1$, they do not contain object boundary
 - Type 2: For all sub-images, if the gray-level variance is $> T2$, they contain object boundary
- ## ➤ All sub-images of type 1 are merged and thresholded using a single threshold value
- ## ➤ Each sub-image of type 2 is thresholded independently

Result



Better?

- How can the result be improved?
- There is no solution that handles all thresholding problems (where more intelligent algorithms are needed)

Variable thresholding based on local image properties

$$Q(\sigma_{xy}, m_{xy}) = \begin{cases} \text{true}, & f(x, y) < a\sigma_{xy} \text{ and } f(x, y) > bm_{xy} \\ \text{false}, & \text{otherwise} \end{cases}$$

$$g(x, y) = \begin{cases} 1, & \text{if } Q \text{ is true} \\ 0, & \text{if } Q \text{ is false} \end{cases}$$



(a) Image from Fig. 10.40. (b) Image segmented using the dual thresholding approach given by Eq. (10-76). (c) Image of local standard deviations. (d) Result obtained using local thresholding.

Basis of Image Segmentation

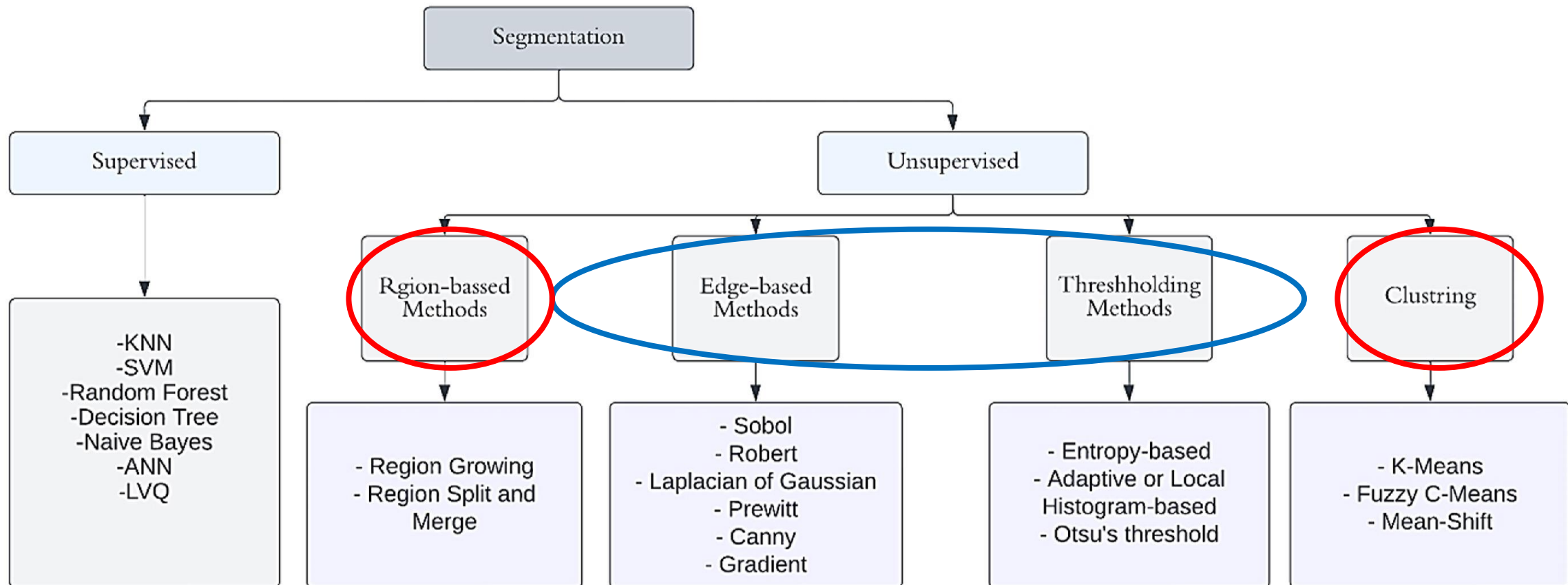
- **Detection of gray level discontinuities**
 - Point detection
 - Line detection
 - Edge detection
 - Gradient operators
 - LoG: Laplacian of Gaussian
- **Thresholding**
 - Global (Otsu's method)
 - Adaptive
 - Using edge characteristics

Image Segmentation

- Two basic properties:
 - Discontinuity – edge detection
 - Detection of Discontinuities
 - Edge Linking and Boundary Detection
 - Similarity
 - Thresholding
 - Region growing, clustering and others



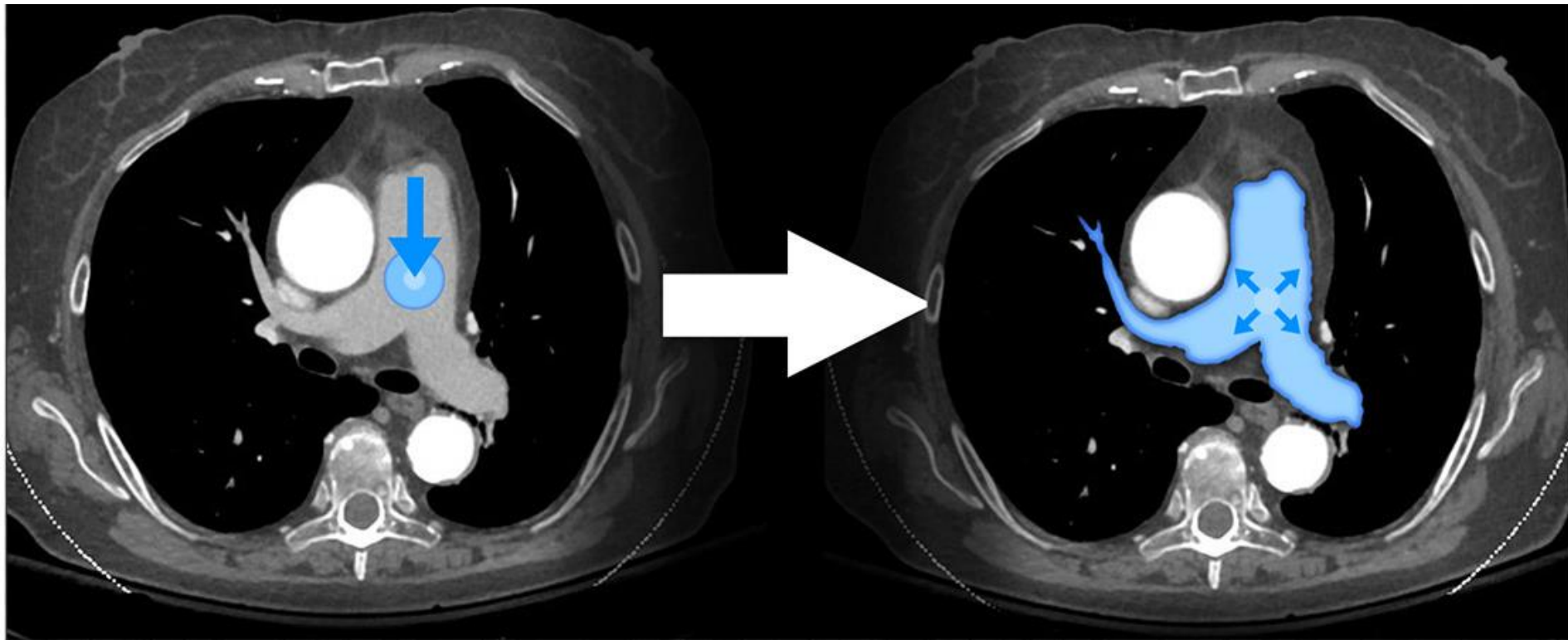
Overview



Region growing

- Groups pixels or subregions into larger regions based on predefined criteria (gray tone or texture)
- Basic method
 - A set of “seed” points
 - Appending to each seed those neighboring pixels that have properties similar to the seed
- Keys in region growing:
 - Selection of the seeds
 - Criteria of similarity (Gray level / connectivity / texture / moments)
 - Connectivity properties
 - Formulation of a stopping rule

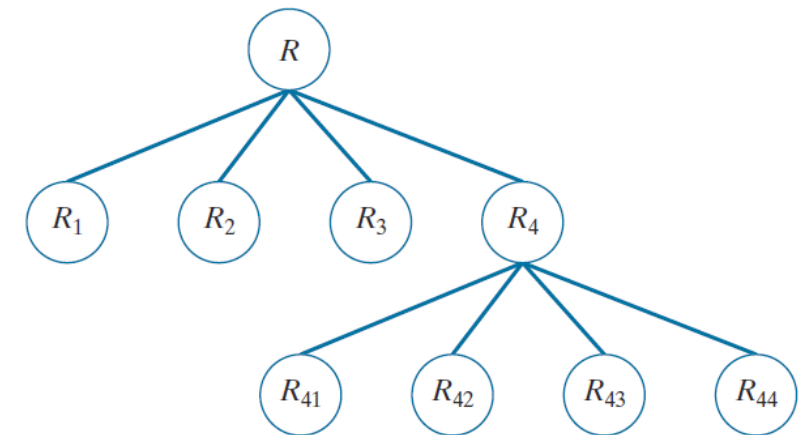
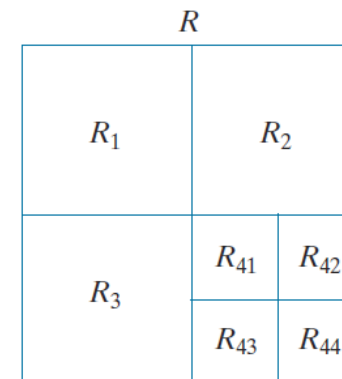
Example



Region Splitting & Merging

➤ Splitting:

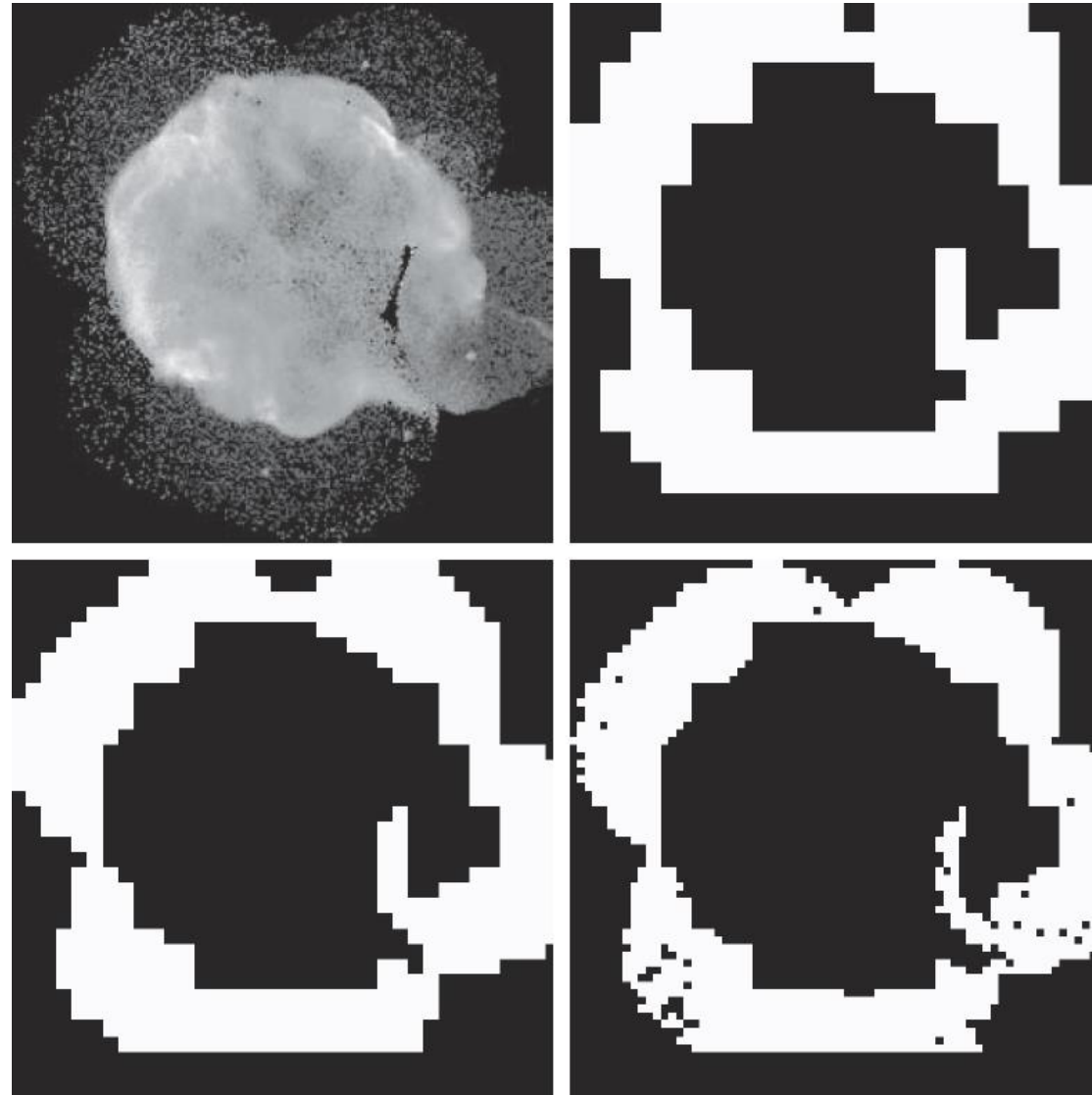
- Starting with the entire region.
- If $Q(R)=FALSE$, divide the image into quadrant
- If Q is FALSE for any quadrant, divide that quadrant into sub-quadrants.
- The result is a quadtree



Region Splitting & Merging

- If only splitting, it is likely that adjacent regions have identical properties.
- Adjacent regions whose combined region satisfies Q should be merged.
- Algorithm
 - **Splitting** into 4 disjoint quadrants if $Q(R)=FALSE$
 - **Merging** any adjacent quadrants for which Q is TRUE
 - Stop when no further merging and splitting is possible

Example



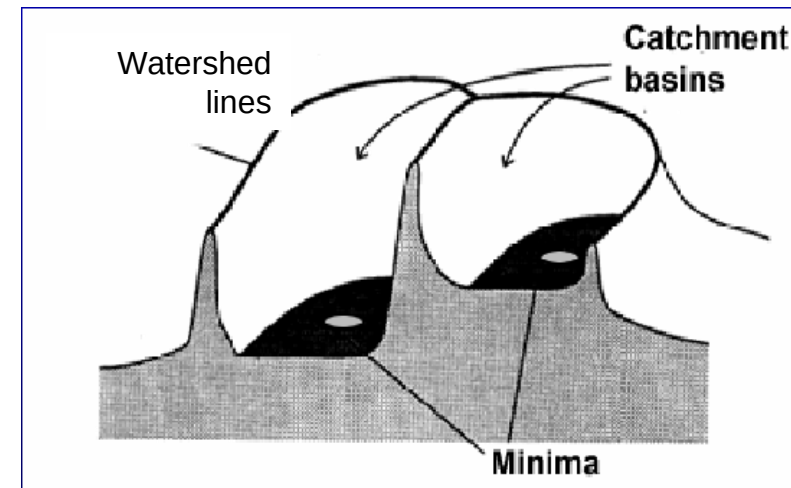
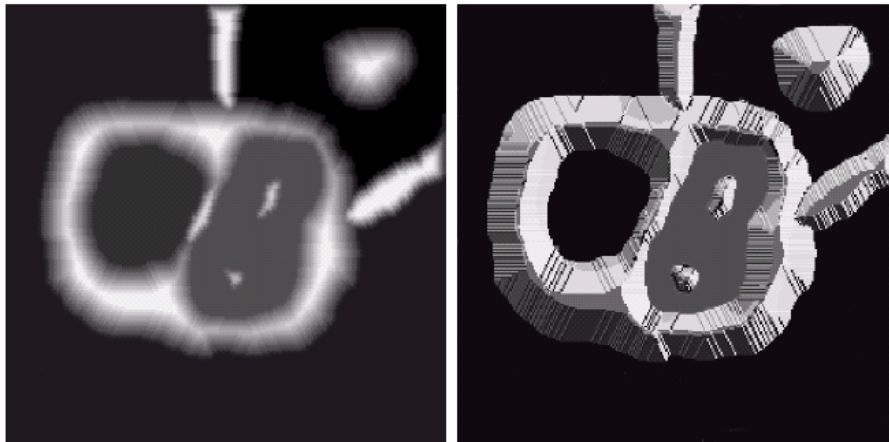
Minimum sub-
quadrants are set to be
32x32

16x16

8x8

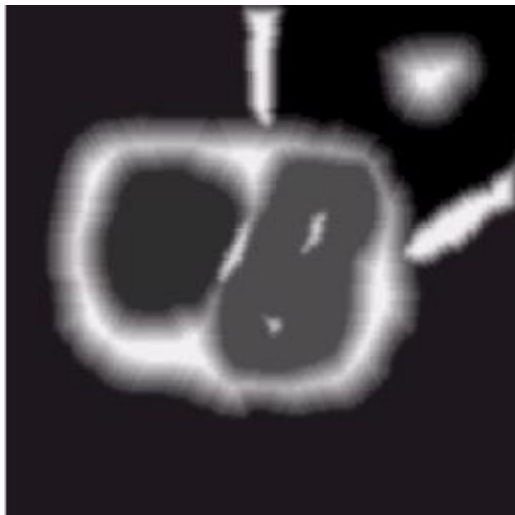
Watershed Segmentation

- Visualize an image in 3D: x, y, and gray levels.
- In such a topographic interpretation, there are 3 types of points:
 - Points belonging to a regional minimum
 - Points at which a drop of water would fall to a single minimum
 - The *catchment basin* or *watershed* of that minimum
 - Points at which a drop of water would be equally likely to fall to more than one minimum.
 - The *divide lines* or *watershed line*

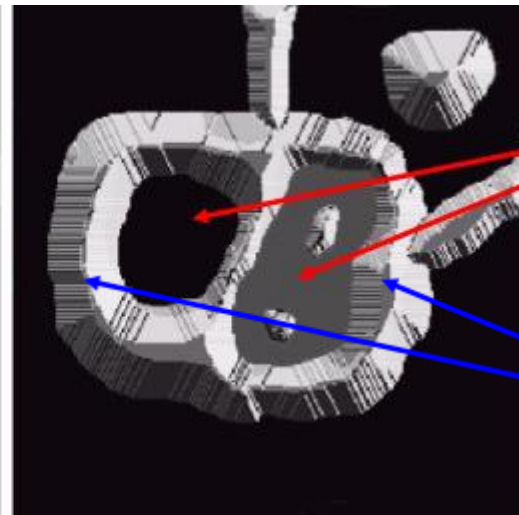


Watershed Segmentation

- Objective: finding the watershed lines
 - Construction of dam to prevent catchment basins from merging
 - Punch a hole in each regional minimal for water to fill up the catchment basin



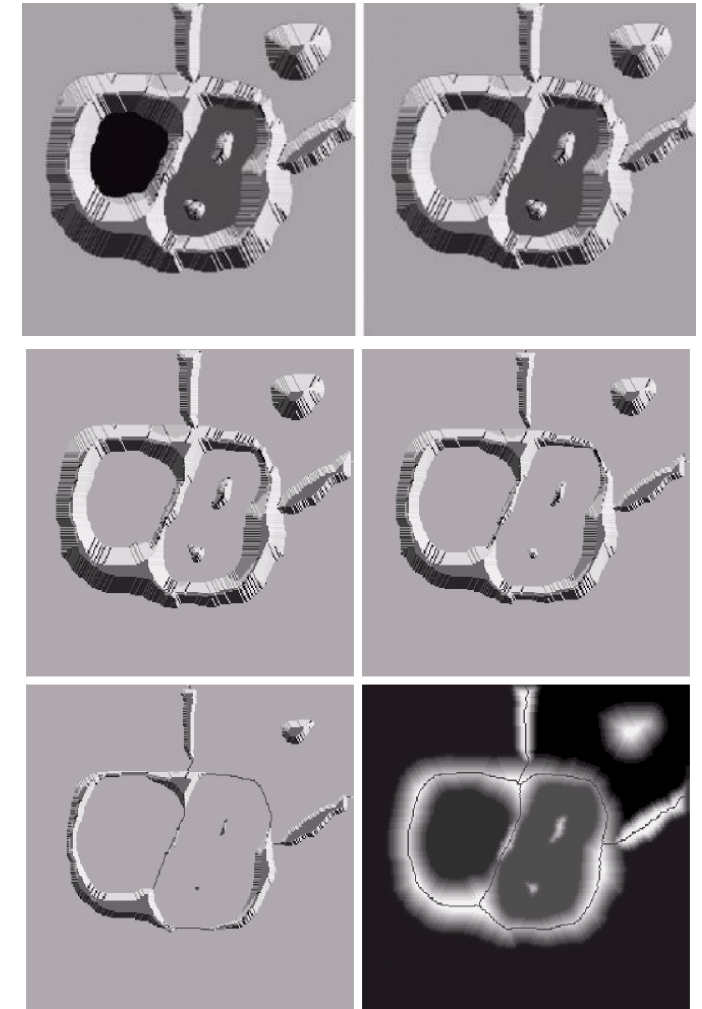
Original image



Topographic view

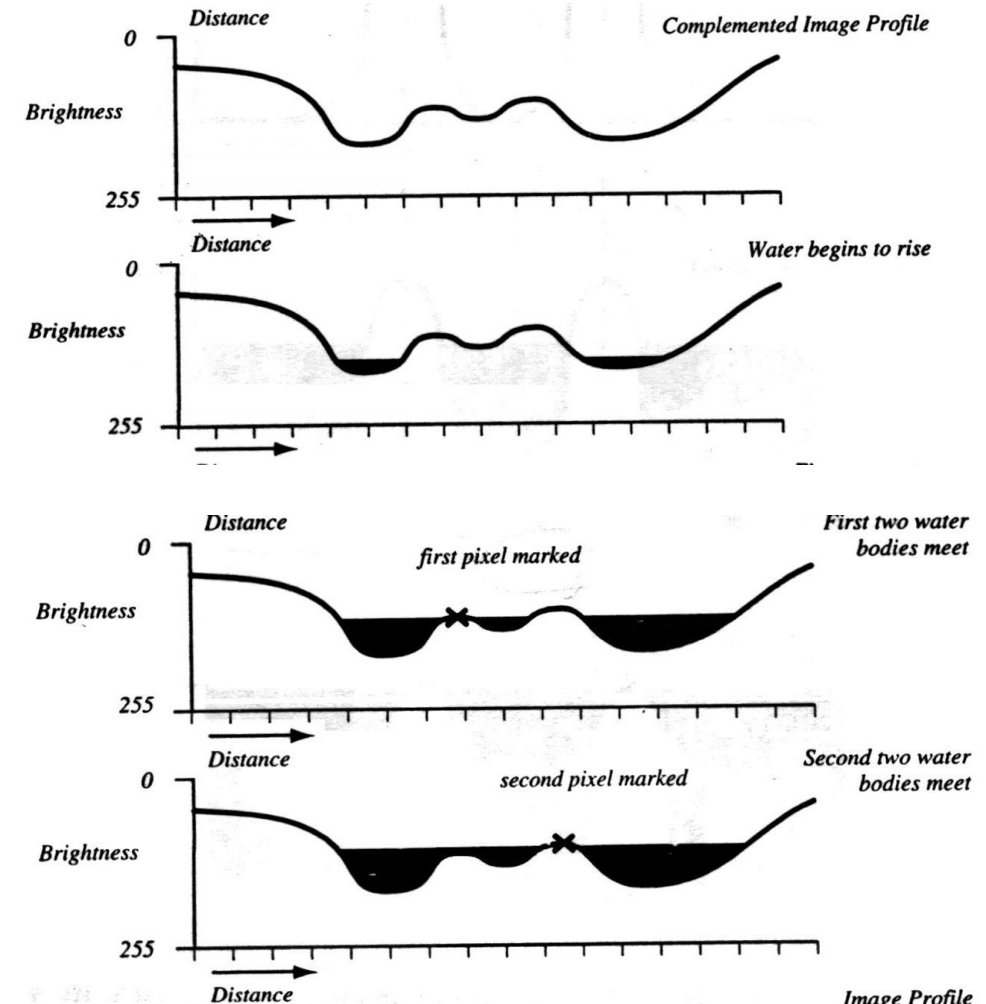
Catchment basins

Watershed lines



Watershed Segmentation

- To find watershed lines.
 - Suppose that a hole is punched in each regional minimum
 - The entire topography is flooded from below by letting water rise through the holes at a uniform rate.
 - When rising water in distinct catchment basins is about the merge, a dam is built to prevent merging.
 - These dam boundaries correspond to the watershed lines.



Watershed Segmentation Algorithm

- Start with all pixels with the lowest possible value.
 - These form the basis for initial watersheds
- For each intensity level k :
 - For each group of pixels of intensity k
 - If adjacent to exactly one existing region, add these pixels to that region
 - Else if adjacent to more than one existing regions, mark as boundary
 - Else start a new region

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

Step 1: Compute pixel list

- 0: (1,3)
- 1:
- 2:
- 3:
- 4:
- 5:
- 6:
- 7:
- 8:
- 9:

pixel list

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

Step 1: Compute pixel list

- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	u	u	u
u	u	u	u	u
u	u	u	u	u
u	u	u	u	u

labels

- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Step 2:
Set all pixels
to “unlabeled”

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	u	u	u
u	u	u	u	u
u	0	u	u	u
u	u	u	u	u

labels

0: (1,3)

1: (3,1), (2,3), (1,4), (4,4)

2: (2,1), (4,1), (1,2), (4,3)

3: (4,0), (3,2), (0,3), (2,4)

4: (3,0), (0,2), (4,2)

5: (2,0)

6: (1,1), (0,4), (3,4)

7: (1,0), (2,2)

8: (0,0), (3,3)

9: (0,1)

pixel list

Step : k=0

Create new catchment
basins

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	u	u	u
u	u	u	u	u
u	0	0	u	u
u	0	u	u	u

labels

(shaded pixels are on frontier)

- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Step : k=1

Grow catchment basins

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	u	u	u
u	u	u	u	u
u	0	0	u	u
u	0	u	u	u

labels

(shaded pixels are on frontier)

- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Step 7: k=1
Expand frontier
(nowhere to go)

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	u	1	u
u	u	u	u	u
u	0	0	u	u
u	0	u	u	2

labels

- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Step 8: k=1
Create new basins

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	1	1	1
u	0	u	u	u
u	0	0	u	2
u	0	u	u	2

labels

(shaded pixels are on frontier)

- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Step 9: k=2
Grow basins

Watershed example

(shaded pixels are on frontier)

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	1	1	1
u	0	u	u	u
u	0	0	u	2
u	0	u	u	2

labels

- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Step 10: k=2
Expand frontier
(nowhere to go)

Watershed example

8	7	5	4	3
9	6	2	1	2
4	2	7	3	4
3	0	1	8	2
6	1	3	6	1

image

u	u	u	u	u
u	u	1	1	1
u	0	u	u	u
u	0	0	u	2
u	0	u	u	2

labels

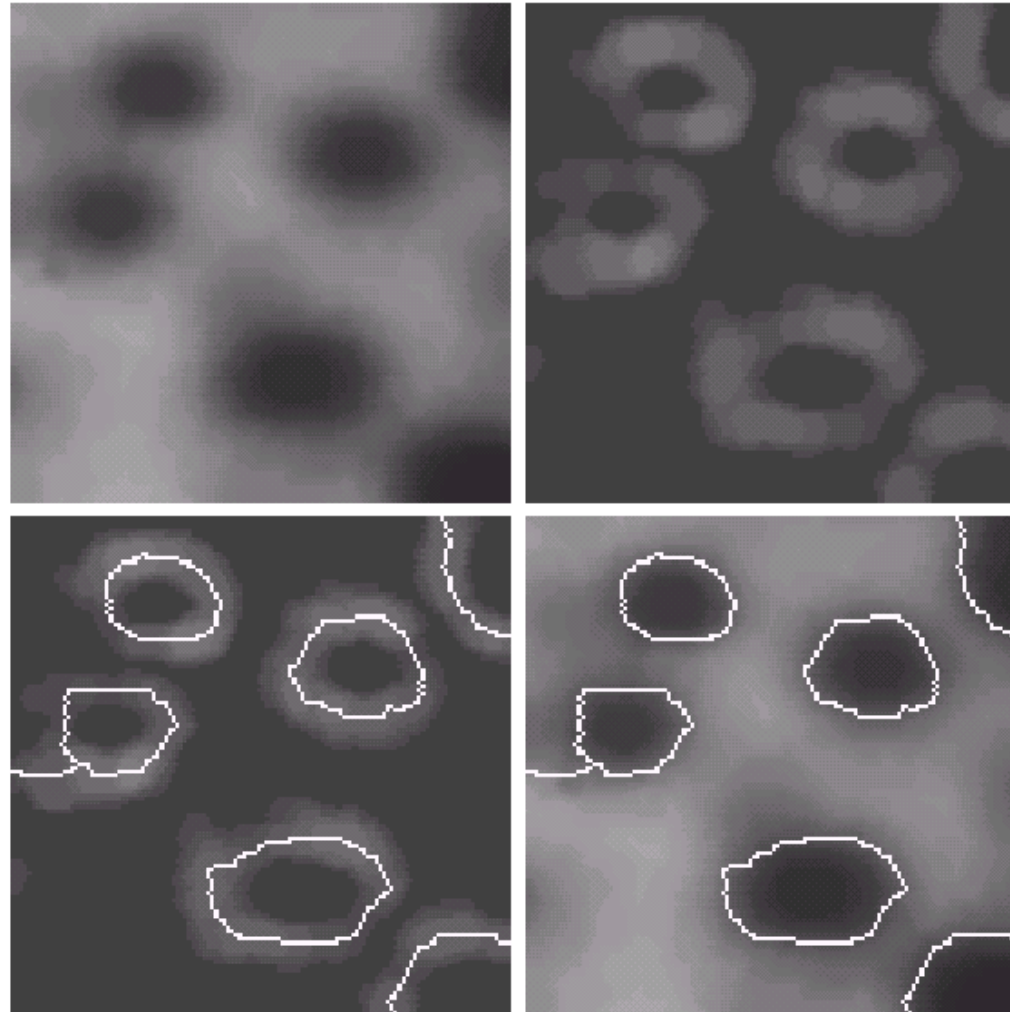
- 0: (1,3)
- 1: (3,1), (2,3), (1,4), (4,4)
- 2: (2,1), (4,1), (1,2), (4,3)
- 3: (4,0), (3,2), (0,3), (2,4)
- 4: (3,0), (0,2), (4,2)
- 5: (2,0)
- 6: (1,1), (0,4), (3,4)
- 7: (1,0), (2,2)
- 8: (0,0), (3,3)
- 9: (0,1)

pixel list

Step 11: k=2
Create new basins
(none to create)

Watershed results

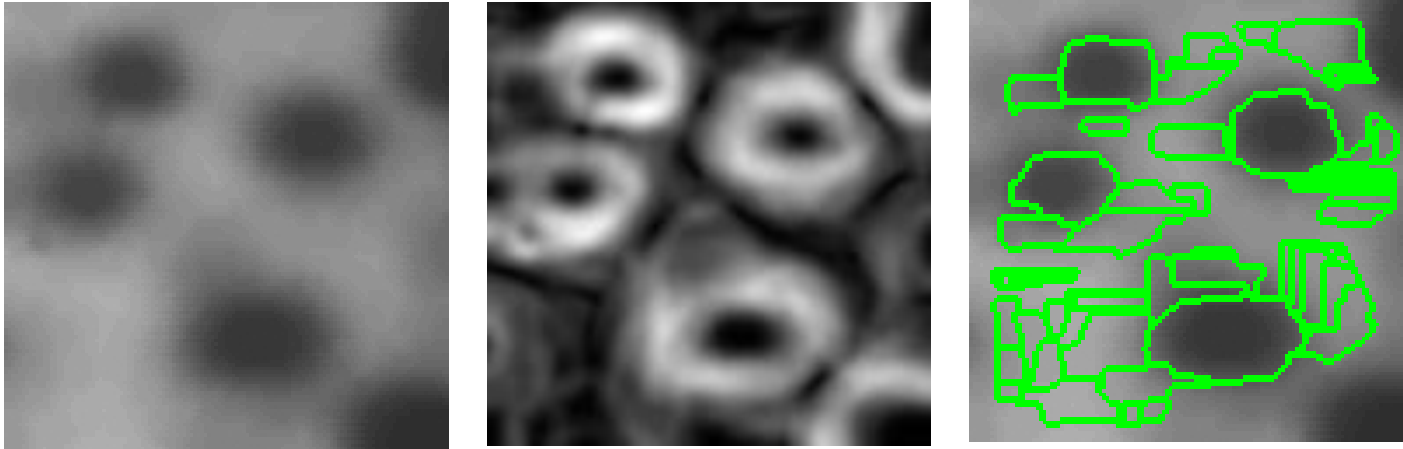
Watershed algorithm might be used on the [gradient image](#) instead of the original image.



gradient image

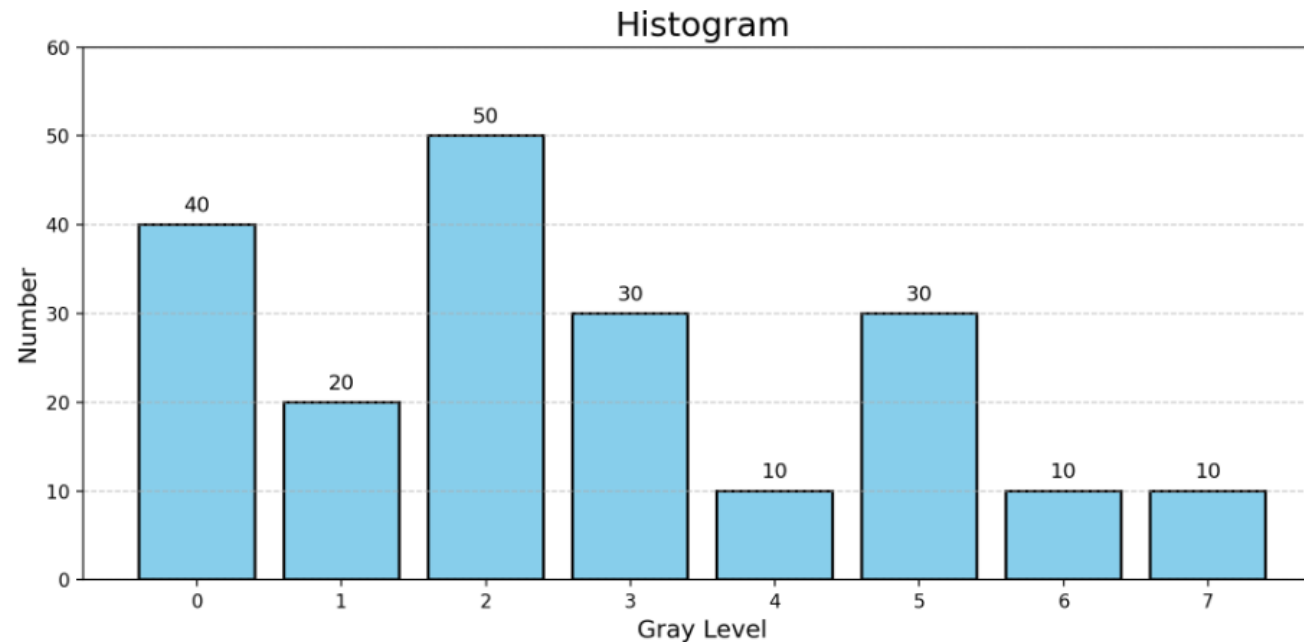
Noise effect on watershed

Due to noise and other local irregularities of the gradient, **oversegmentation** might occur.



Homework1

- A grayscale image has the following histogram:



- Use **Otsu's method** to compute the optimal threshold that maximizes the between-class variance.

Homework2

- Divide the image into four non-overlapping 8 x 8 sub-images, and analyze the local threshold for each sub-image.
- Write down the complete 16 x 16 binary segmentation result, and determine whether a single global threshold can produce the same result.

0.0	0.5	0.0	0.5	0.0	0.5	0.0	0.5	1.0	1.5	1.0	1.5	1.0	1.5	1.0	1.5
0.5	0.0	0.5	2.0	2.5	2.0	2.5	2.0	3.5	3.0	3.5	3.0	3.5	1.0	1.5	1.0
0.0	0.5	0.0	2.5	2.0	2.5	2.0	2.5	3.0	3.5	3.0	3.5	3.0	1.5	1.0	1.5
0.5	0.0	0.5	0.0	0.5	0.0	0.5	0.0	1.5	1.0	1.5	3.0	3.5	1.0	1.5	1.0
0.0	0.5	0.0	0.5	0.0	0.5	0.0	0.5	1.0	1.5	1.0	3.5	3.0	1.5	1.0	1.5
0.5	0.0	0.5	0.0	0.5	0.0	0.5	0.0	1.5	1.0	1.5	3.0	3.5	1.0	1.5	1.0
0.0	0.5	0.0	0.5	0.0	0.5	0.0	0.5	1.0	1.5	1.0	3.5	3.0	1.5	1.0	1.5
0.5	0.0	0.5	2.0	2.5	2.0	2.5	2.0	3.5	3.0	3.5	3.0	3.5	1.0	1.5	1.0
2.0	2.5	2.0	4.5	4.0	4.5	4.0	4.5	5.0	5.5	5.0	5.5	5.0	3.5	3.0	3.5
2.5	2.0	2.5	4.0	4.5	2.0	2.5	2.0	3.5	3.0	3.5	3.0	3.5	3.0	3.5	3.0
2.0	2.5	2.0	4.5	4.0	2.5	2.0	2.5	3.0	3.5	3.0	3.5	3.0	3.5	3.0	3.5
2.5	2.0	2.5	4.0	4.5	2.0	2.5	2.0	3.5	3.0	3.5	3.0	3.5	3.0	3.5	3.0
2.0	2.5	2.0	4.5	4.0	2.5	2.0	2.5	3.0	3.5	3.0	3.5	3.0	3.5	3.0	3.5
2.5	2.0	2.5	4.0	4.5	4.0	4.5	4.0	5.5	5.0	5.5	5.0	5.5	3.0	3.5	3.0
2.0	2.5	2.0	4.5	4.0	4.5	4.0	4.5	5.0	5.5	5.0	5.5	5.0	3.5	3.0	3.5
2.5	2.0	2.5	2.0	2.5	2.0	2.5	2.0	3.5	3.0	3.5	3.0	3.5	3.0	3.5	3.0

Homework3

Given a 5 x 5 grayscale image I as follows:

$$I = \begin{bmatrix} 50 & 55 & 60 & 65 & 70 \\ 55 & 60 & 70 & 75 & 80 \\ 60 & 70 & 150 & 160 & 90 \\ 65 & 75 & 160 & 170 & 100 \\ 70 & 80 & 90 & 100 & 110 \end{bmatrix}$$

Please apply the main steps of the edge detection algorithm to extract edges from the image. The specific requirements are as follows:

- Compute Gradients: Use the 3 x 3 Sobel operator to compute the gradients G_x (horizontal) and G_y (vertical) of the image I . Apply zero-padding to handle the boundaries.
- Compute Gradient Magnitude and Direction: Based on G_x and G_y , calculate the gradient magnitude $\sqrt{G_x^2 + G_y^2}$ and the gradient direction $\theta = \arctan(G_y/G_x)$ for each pixel.